

Safe and Energy-Efficient Trajectory Planning for Heterogeneous Multi-UAV Enabled Mobile Edge Computing

Xiuling Zhang^{1b}, Riheng Jia^{1b}, *Member, IEEE*, Qianjun Yin^{1b}, Zhonglong Zheng^{1b}, *Member, IEEE*, and Minglu Li^{1b}, *Fellow, IEEE*

Abstract—Mobile edge computing (MEC) has recently gained significant attention as a promising solution for processing delay-sensitive and resource-intensive computational jobs. Existing system schedulers in MEC networks typically assume homogeneous service providers, uniformly distributed user equipment (UE), and identical service requirements, making them unsuitable for practical MEC scenarios where jobs are randomly generated with varying service and completion time requirements. Thus, in this work, we jointly optimize job scheduling and resource allocation in a heterogeneous multi-unmanned aerial vehicle (UAV) enabled MEC network, considering practical factors such as diverse service requirements of jobs, unknown distribution of UEs, and spatial-temporal job arrivals. We aim to reduce the overall job miss rate and the average energy consumption of both UAVs and UEs by jointly planning safe UAV trajectories and onboard resource allocation. To learn uncertain and dynamic UE-side states (e.g., job arrivals and mobility patterns) and ensure the UAV's safety during the flight, we propose a multi-agent safe reinforcement learning algorithm that combines a Shared Soft Actor-Critic architecture for extracting features of heterogeneous UAVs and a two-agent Markov Game of Intervention mechanism for collision avoidance, named SSAC-MGI. In particular, SSAC-MGI further incorporates a fine-grained resource allocation scheme to improve onboard resource utilization and reduce job miss rate. Extensive real trace-driven simulations based on Alibaba cluster data validate the effectiveness and superiority of SSAC-MGI, compared with several state-of-the-art algorithms.

Index Terms—Mobile edge computing, multi-UAV system, cooperative trajectory planning, safe reinforcement learning, job scheduling, resource allocation.

I. INTRODUCTION

A. Background

THE proliferation of edge-based machine learning applications, such as natural language processing and intelligent

question answering, has led to a surge in the creation and utilization of execution workflows at the network edge [1]. These workflows usually require dedicated computational resources, such as pre-deployed edge compute instances [2], to meet stringent real-time requirements. However, the constraints of limited computational capacity and battery life of user equipment (UE) render the local execution of such workflows impractical. In addition, the dynamic and unpredictable distribution of UEs (e.g., due to the mobility), combined with the diverse and time-varying computational demands, poses substantial challenges for statically deployed edge servers. Recent advances in unmanned aerial vehicle (UAV) technologies, encompassing autonomous navigation, infrastructure monitoring, and onboard computing [3], have catalyzed the emergence of UAV enabled mobile edge computing (MEC) as a promising paradigm for delivering workflows directly at the network edge [4], [5], [6], [7], [8].

UAVs acting as MEC platforms can support agile mobility and achieve high probability of line-of-sight (LoS) communication, thereby offering enhanced deployment flexibility and wide-area coverage tailored to the computational demands at the network edge. Furthermore, recent advances in distributed training and inference of multi-modal foundation models over next-generation (6G) wireless networks [9] provide a system foundation for deploying intelligent services at the wireless edge, including UAV-enabled MECNs. *Despite these advantages, several inherent limitations hinder the full potential of UAV-assisted MECNs:* (i) the UAV's limited battery capacity constrains its service duration; (ii) the UAV's limited onboard storage imposes constraints on both the diversity and quantity of pre-deployed computational resources; and (iii) the dynamic and unpredictable distribution of UEs, the time-varying and diverse computational demands, and the tight coupling between job scheduling and resource allocation together add the complexity to UAV enabled MEC problems. Although UAV-assisted MECNs have been widely studied [8], [10], [11], [12], [13], most existing work either focus on a single UAV or treat multiple UAVs as independent entities [14]. In fact, effective collaboration among UAVs especially when they have heterogeneous service capabilities, can significantly improve the job scheduling and resource utilization, thus enhancing the quality of service (QoS) for UEs.

Received 17 May 2025; revised 3 November 2025; accepted 11 November 2025. Date of publication 14 November 2025; date of current version 6 March 2026. This work was supported by the National Natural Science Foundation of China under Grant 62272417, Grant 62272419, and Grant 62320106006. Recommended for acceptance by Y. Zeng. (*Corresponding author: Riheng Jia.*)

Xiuling Zhang and Qianjun Yin are with the College of Systems Engineering, National University of Defense Technology, Changsha 410073, China (e-mail: xiuling@nudt.edu.cn; yin_qianjun@163.com).

Riheng Jia, Zhonglong Zheng, and Minglu Li are with the School of Computer Science and Technology, Zhejiang Normal University, Jinhua 321004, China (e-mail: rihengjia@zjnu.edu.cn; zhonglong@zjnu.edu.cn; mlli@zjnu.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3632884

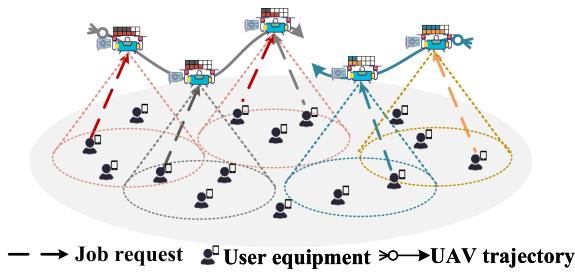


Fig. 1. Multi-UAV enabled MECN model.

B. A Brief Introduction of Our Work and Challenges

In this work, we consider a dynamic and stochastic multi-UAV enabled MECN, where both the locations of UEs¹ and the arrivals of job requests at UEs are unknown to UAVs in advance. Each UE generates job requests associated with specific service types and resource requirements, and the aggregate job requests arrivals across the network follow a Poisson process. Considering the UAV's limited onboard storage capacity, it is unrealistic for each UAV being equipped with all types of computing services. Thus, we assume that each UAV is heterogeneous regarding its pre-deployed service types and resource capacity designated for executing instances of each service type. For example, in Fig. 1, each UAV is equipped with two service types (represented by different colors), and provisioned with dedicated resources to execute workflow instances of the corresponding types. While multiple heterogeneous UAVs enable the broad service coverage and rich computational resources, they also pose significant challenges such as collision-free trajectory planning and service-constrained job scheduling (i.e., UEs may have to find an appropriate UAV with their required service types to upload job requests), especially when the MECN has job arrivals with spatial-temporal dynamics and diverse service demands. To this end, we propose a safe and cooperative trajectory planning method for multiple heterogeneous UAVs, with the objective of reducing the job miss rate and the average energy consumption of both UAVs and UEs. *We summarize the main challenges of solving this problem as follows:*

- *Heterogeneity of UAVs in service types and resource capacities:* The constrained service types and resource capacities on each UAV cause the UAV to fly back and forth within the network to collect the appropriate job requests to serve, which significantly increases the flight energy consumption. Conversely, designing energy-conserving trajectories can reduce the flight energy consumption, which however may increase the job waiting time on the UE side. This trade-off needs to be carefully handled for achieving both low energy consumption and timely job completion in UAV enabled MECNs.
- *Job arrivals with spatial-temporal dynamics and diverse service demands:* Since job requests are randomly generated at each UE and UEs may randomly move within the network, the job arrivals across the entire network present

spatial-temporal dynamics, which are difficult to predict. Thus, it is challenging to safely and efficiently schedule the UAVs in such a complex, dynamic and unpredictable MEC environment to achieve high QoS for UEs, especially when considering the strict and diverse completion deadlines of different jobs.

- *Coupled multiple optimization objectives:* The optimization objectives in this work, i.e., the job miss rate, the average energy consumption of both UAVs and UEs, and the overall flight safety, are interdependent and conflicting, therefore making the cooperative trajectory planning challenging. For example, to avoid the mutual collision, UAVs may need to frequently alter their flight routes (e.g., deviate from their optimal routes), which however may increase the energy consumption or the job miss rate.

To address the aforementioned challenges, we propose a multi-agent safe reinforcement learning algorithm that integrates a Shared Soft Actor-Critic (SSAC) architecture for extracting UAV-specific heterogeneous features and a two-agent Markov Game of Intervention (MGI) for collision avoidance, named SSAC-MGI. By leveraging consistency in certain onboard resource dimensions, such as service types, available CPU, and memory, among heterogeneous UAVs, we construct a set of shared policy networks that encode their common characteristics into high-level and transferable skills, thereby reducing the policy complexity associated with trajectory planning. We model the interaction between heterogeneous UAVs and the MECN environment as a decentralized partially observable Markov decision process (Dec-POMDP), and adopt a decentralized training and decentralized execution (DTDE) framework, where each UAV independently makes real-time flight decisions based solely on its local observations, while the policies are optimized to maximize global system rewards. *The main contributions of this work are as follows:*

- We propose a novel framework that jointly optimizes safety-aware and energy-efficient trajectory for service provisioning in heterogeneous multi-UAV enabled MECNs, handling dynamic arriving, delay-sensitive UE requests, without requiring inter-UAV communication. The problem is formulated as a two-agent Dec-POMDP, which is solved by proposing a novel multi-agent safe reinforcement learning algorithm called SSAC-MGI.
- SSAC-MGI incorporates a two-agent MGI mechanism, where each UAV is jointly controlled by a safety agent and a standard agent. Specifically, the safety agent aims to prevent the UAV from colliding with other UAVs or obstacles during the mission, while the standard agent aims to improve the UAV's mission performance by minimizing the job miss rate and reducing the average energy consumption. In contrast to traditional approaches that incorporate safety via reward shaping, our method explicitly decouples safety control from task performance optimization, thereby improving the training convergence and enabling a better trade-off between safety and task performance in UAV enabled MECNs.
- We design a *Round-Robin* instance allocation method for assigning computational instances to workflow queues on

¹UEs can be either *static* with random spatial distribution or *mobile* with uncertain mobility trajectories.

TABLE I
KEY DIFFERENCES BETWEEN OUR WORK AND MAIN RELEVANT LITERATURE

Literature	Dynamic Requests ¹	Inaccessible ²	Heterogeneous ³	Learning based	Optimization Goals
[5], [6], [11], [15]	✗	✗	✗	✗	Energy efficiency, completion time
[10], [16]	✗	✓	✗	✓	Sum transmit rate, quality of experience (QoE)
[7], [17]	✓ (Random)	✗	✓ (Resource)	✗	QoE, service latency
[18], [19]	✗	✓	✓ (Parameters)	✓	Energy efficiency
[20], [21]	✗	✓	✗	✓	Fairness throughput
[12], [13]	✓ (Continuous)	✗	✗	✓	Energy efficiency, service latency
Our work	✓ (Random)	✓	✓ (Resource)	✓	Energy efficiency, safety, job miss rate

¹ Continuous and random indicate that UEs generate requests either continuously or at random intervals during the system operation time.

² Inaccessible refers to the scenario where UEs' locations are unknown (including the case when UEs are mobile).

³ Heterogeneous indicates the differences in task demands and UAV-provided service types (e.g., computing resources) or system parameters.

UAVs. This method enhances resource utilization and improves the likelihood that each workflow is completed before its deadline. We conduct simulation experiments in a multi-UAV enabled MECN environment constructed based on real-world data, including UE locations from Twitter and workflow traces from Alibaba. Extensive evaluations across diverse scenarios demonstrate the superiority and robustness of SSAC-MGI, compared with several state-of-the-art approaches.

II. RELATED WORK

In this section, we provide the literature review on UAV trajectory planning in MECNs. In particular, Table I summarizes the key differences between our work and main relevant work.

A. Non-Learning Based UAV Trajectory Planning in MECNs

In the following, we review existing studies on optimization scenarios involving single-UAV solutions and multi-UAV cooperative approaches, with a particular focus on methods that transform non-convex problems into convex formulations.

Trajectory Planning for a Single UAV: To address the problem that severely blocked UEs cannot fully utilize the computing resources of access points (APs), Hu et al. [5] investigated a UAV-assisted MEC architecture that enables the simultaneous utilization of computing resources at both the UAV and the AP. They proposed an alternating optimization algorithm that jointly optimizes the computation scheduling, bandwidth allocation, and UAV trajectory to minimize the weighted sum of energy consumption of the UAV and UEs, under several practical constraints. Further, Li et al. [6] employed the successive convex approximation technique together with the Dinkelbach algorithm to maximize the UAV's energy efficiency by jointly optimizing the UAV's trajectory, user transmission power, and computation load allocation. Similarly, Hu et al. [15] developed a multi-stage alternating optimization algorithm based on the Dinkelbach method and block coordinate descent (BCD) method to jointly optimize the computation scheduling, transmission power control, and 3D UAV trajectory.

Trajectory Planning for Multiple UAVs: Since a single UAV is constrained in energy and computation capacity, which may not satisfy users' service demands especially in large-scale MECNs,

cooperative multi-UAV enabled MEC systems have gained increasing attention for their flexibility, scalability, and enhanced task processing capability. For example, Zhou et al. [17] proposed a comprehensive optimization framework to minimize the service latency taking into account unique characteristics of UAVs. By leveraging the Lyapunov optimization technique and a dependent rounding technique, the proposed framework jointly determines the cache placement, UAV trajectories, UE-UAV associations, and task offloading, subject to the energy and resource constraints of UAVs. Similarly, the authors in [7] investigated the joint optimization of task offloading, resource allocation, trajectory planning, and service cache placement under energy and delay constraints, and the goal is to reduce the service latency while ensuring the fairness among UEs.

B. Learning Based UAV Trajectory Planning in MECNs

Traditional non-learning based methods rely on exact system model and entail high computational complexity, limiting their applicability in solving large scale or highly dynamic problems. Given the highly complex and dynamic MEC environment, deep reinforcement learning (DRL) has emerged as a promising solution due to its adaptability and performance advantages. For example, Liu et al. [10] proposed a multi-agent Q-learning based algorithm for joint trajectory design and power control, leveraging predicted user mobility to maximize the instantaneous sum transmission rate while satisfying the rate requirement of users. Similarly, to handle the high-dimensional and continuous action space, Zhao et al. [12] proposed a cooperative multi-agent DRL algorithm based on TD3, which jointly optimizes UAV trajectories, task offloading, and transmission power to minimize total system costs. Hao et al. [22] investigated task offloading in a UAV-assisted MEC system with multi-UAV collaboration, highlighting task priorities and a binary offloading mode. Song et al. [21] investigated a multi-objective proximal policy optimization algorithm for joint trajectory control and task offloading, aiming to minimize the task delay and UAVs' energy consumption and maximize the number of collected tasks. Liu et al. [13] proposed a decentralized DRL based framework to control each UAV in a distributed manner, aiming to maximize the average coverage score while minimizing the energy consumption. Wang et al. [18] proposed the soft hierarchical DRL network and dual-end federated reinforcement learning as

decentralized navigation policy frameworks to address the UAV heterogeneity characterized by varying system parameters.

Compared with most existing work on MECNs including the above mentioned literature, *the novelty of this work is summarized as follows*: (i) unlike methods in [18], [19] that assigned each UAV to independently train a policy network for a specific spatially partitioned cluster of users, we collaboratively train all heterogeneous UAVs to jointly serve the entire network, thereby better accommodating the diverse service demands with spatial-temporal dynamics; (ii) in contrast to previous studies such as [13], [23], which adopted the centralized training and decentralized execution (CTDE) paradigm, we employ the SSAC architecture and train it with DTDE, thus avoiding the significant communication and computational overhead caused by transmitting and integrating all UAV observations; (iii) most existing DRL methods, such as [22], were unable to learn multiple policies in a single training process to accommodate diverse objective preferences among heterogeneous UAVs. In this work, we adopt a policy-sharing paradigm to facilitate knowledge sharing among heterogeneous UAVs with varying features along the same dimensions, thereby enabling the learning of a unified policy.

III. SYSTEM MODEL

As shown in Fig. 1, we consider a heterogeneous multi-UAV-enabled MECN comprising M static or mobile UEs, denoted by the set $\mathcal{M} = \{1, \dots, M\}$, and N rotary-wing UAVs, denoted by the set $\mathcal{N} = \{1, \dots, N\}$. The network heterogeneity arises from two aspects: 1) each UE randomly generates job requests with diverse resource demands during network operation; 2) each UAV hosts a subset of all available service types, thereby introducing the inter-UAV resource heterogeneity. Job requests are randomly generated by the UEs. Each job, which contains the input data and workflow execution description, is scheduled to be processed by certain UAVs. Given the limited storage capacity of UAVs and highly diverse service demands of UEs, each UAV is assumed to be pre-configured with a subset of the available computational instance types, representing the available service types. The instance type and the corresponding quantity may differ in different UAVs, and the union of instance types across all UAVs can cover the full range of the request types generated by all UEs. We assume a time-slot model $\mathbb{T} = \{1, 2, \dots, T\}$, where each time slot has an equal time interval. During the MECN operation, UAVs dynamically plan their trajectories based on service type constraints and available resources to efficiently schedule and process jobs. Table II summarizes the main system parameters.

A. Job Request Model

Based on the time-slot model, the three-dimensional (3D) coordinate of UE m at time slot t is denoted by $l_m^t = (x_m^t, y_m^t, 0)$, $\forall m \in \mathcal{M}$, $t \in \mathbb{T}$. In the following, we describe the job in detail by analyzing its generation, composition and execution.

1) *Job Generation*: To model the random job generation process, we assume that, in each time slot, the number of jobs generated by all UEs in the MECN follows a Poisson distribution

TABLE II
ILLUSTRATION OF MAIN SYSTEM PARAMETERS

Symbols	Definitions
$m \in \mathcal{M}$	Index of UEs
$n \in \mathcal{N}$	Index of UAVs
$t \in \mathbb{T}$	Index of time slots
$\gamma_m^t \subset \Gamma$	A set of jobs generated and accumulated by UE m at t
$j_k \in \gamma_m^t$	A job indexed by k
$\mathcal{R}_n \subset \mathcal{R}$	The set of resource types deployed on UAV n
$r \in \mathcal{R}_n$	Index of resource types deployed on UAV n
$r_k \in \mathcal{R}$	The resource type required by job j_k
v_k^i	The i -th task within job j_k
t_k^g	Generation time of j_k
ϕ_k^i	Successful execution duration of task v_k^i in j_k
ξ	The deadline tightness factor of tasks
$\Phi_{m,k}^t$	The job request of j_k from UE m
t_k^c	The latest completion timestamp of j_k
$t_{k,i}^e$	The actual execution timestamp of v_k^i
$t_{k,i}^f$	The final completed timestamp of v_k^i
$d_{m,n}^{hori,t}$	The horizontal distance between UE m and UAV n
$h_{m,n}^t$	The channel gain between UE m and UAV n
Ω_k	Data size of job j_k formed as a request file
p_m^t	The transmission power of user m for uploading
$P_{m,n}^{LoS}$	The LoS probability between UE m and UAV n
$P_{m,n}^{NLoS}$	The NLoS probability between UE m and UAV n
$y_{n,k}^t$	Whether job j_k is successfully uploaded to UAV n at t
$\mathbf{I}_n^t$	Idle state of UAV n at t (idle is 1 or busy is 0)
$\mathcal{D}_n^t$	Queue of job requests received by UAV n at t
$vcpu_n^r$	Capacity of type- r CPU cores in UAV n
$vmem_n^r$	Capacity of type- r memory in UAV n
$x_{v_k^i, vm_n^r}^t$	Number of type- r computation instances allocated to task v_k^i on UAV n at time slot t
p_n^t	The flight power of UAV n at time slot t

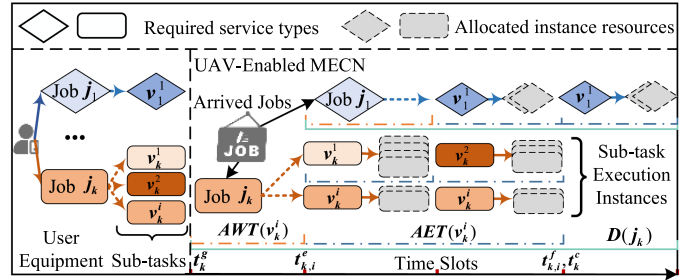


Fig. 2. Job request and process model.

with rate parameter λ . The set of jobs that are newly generated and historically accumulated at UE m at location l_m^t is denoted by $\gamma_m^t = \{j_1, j_2, \dots, j_k\}$, where j_k denotes the job with index k . The set of all newly generated and accumulated jobs in the MECN at time slot t is defined as $\Gamma = \{\gamma_1^t, \gamma_2^t, \dots, \gamma_M^t\}$, $\forall t \in \mathbb{T}$. Let t_k^g denote the generation time of job j_k at the corresponding UE. When UEs send job requests, they provide their current locations, job generation time, and workflow details.

2) *Job Composition*: As shown in the left part of Fig. 2, each job j_k may comprise multiple independent sub-tasks (represented by different colors), all of which require the same service type. In the rest of this paper, we use “task” to denote “sub-task” for simplicity. We define r_k as the type of

computational resource required by all tasks within job j_k . According to the Alibaba cluster trace [24], each task is executed via finer-grained computational instances, which are regarded as the smallest atomic units of execution [2]. The i -th task within job j_k is denoted by v_k^i . The resources needed to execute a single instance of task v_k^i include CPU cores and memory, denoted by $vcpu_k^i$ and $vmem_k^i$, respectively. The UAV, as the executor of the job, can specify the degree of parallelism for each task, which is represented by the number of instances num_k^i . The successful execution time of a task with a parallelism degree of num_k^i is denoted as ϕ_k^i . Therefore, the metadata for task v_k^i can be represented as a tuple $\{(vcpu_k^i, vmem_k^i), num_k^i, \phi_k^i\}$. A job request is formally defined as $\Phi_{m,k}^t = \langle l_m^t, t_k^g, r_k, \{(vcpu_k^i, vmem_k^i), num_k^i, \phi_k^i\}_{\forall v_k^i \in j_k} \rangle$, where the job generation time satisfies $t_k^g \leq t$.

3) *Job Execution Timeline*: Each job can be scheduled to any UAV equipped with the required service type. As shown in the right part of Fig. 2, the actual execution timestamp $t_{k,i}^e$ of each task v_k^i in job j_k is defined as the time when the UAV begins allocating instances for the task. The execution ends when the task is fully completed at timestamp $t_{k,i}^f$. Therefore, the actual execution time (AET) is defined as $AET(v_k^i) = t_{k,i}^f - t_{k,i}^e$. Prior to execution, the task may either wait in the job queue at the UE side or remain in the UAV's request queue. The actual waiting time (AWT) is given by $AWT(v_k^i) = t_{k,i}^e - t_k^g$. Thus, the actual finish time (AFT) of v_k^i is given by

$$AFT(v_k^i) = AWT(v_k^i) + AET(v_k^i). \quad (1)$$

The completion timestamp t_k^c of job j_k is given by

$$t_k^c = t_k^g + \max \{AFT(v_k^i), \forall v_k^i \in j_k\}. \quad (2)$$

In this work, we assume that a job is successfully completed only if all its constituent tasks are completed within their allocated time budgets. Specifically, each task v_k^i is associated with a time budget $\xi \phi_k^i$, where ϕ_k^i denotes its minimum required execution time and $\xi \geq 1$ reflects the allowed slackness [2]. The success of a task requires satisfying the following deadline constraint

$$t_k^g + AFT(v_k^i) \leq t_k^g + \xi \phi_k^i, \forall v_k^i \in j_k, \quad (3)$$

and the success of a job requires satisfying the following constraint

$$t_k^c \leq D(j_k) := \max_{v_k^i \in j_k} \{t_k^g + \xi \phi_k^i\}, \quad \forall j_k \in \Gamma, \quad (4)$$

where $D(j_k)$ represents the deadline of job j_k .

B. UAV-UE Communication Model

The position of the n -th UAV at time slot t is given by $l_n^t = (x_n^t, y_n^t, H)$, $\forall n \in \mathcal{N}, t \in \mathbb{T}$, where H is the height of the UAV and is assumed to be a constant. For UE m , its horizontal distance to UAV n is given by

$$d_{m,n}^{hori,t} = \sqrt{(x_m^t - x_n^t)^2 + (y_m^t - y_n^t)^2}, \quad \forall m \in \mathcal{M}, n \in \mathcal{N}, t \in \mathbb{T}. \quad (5)$$

The prerequisite for successful communication between UAV and UE is that UE m must be located within the coverage area of UAV n [25], that is

$$d_{m,n}^{hori,t} \leq Z, \forall j_k \in \gamma_{m,n}^t, m \in \mathcal{M}, n \in \mathcal{N}, t \in \mathbb{T}, \quad (6)$$

where $Z = H \tan \theta$ is the coverage radius of UAV n .

In this communication model, the communication link between the UE and UAV is referred to as ground-to-air (GTA) link. Specifically, the probability of line-of-sight (LoS) transmission between UE m and UAV n is defined as

$$P_{m,n}^{LoS} = \frac{1}{1 + \rho e^{-\beta[\theta_{m,n} - \rho]}}, \quad (7)$$

where ρ and β represent environmental parameters, while $\theta_{m,n} = \frac{180}{\pi} \sin^{-1}(\frac{H}{d_{m,n}^{elev}})$ and $d_{m,n}^{elev} = \sqrt{(d_{m,n}^{hori})^2 + H^2}$ denote the elevation angle (in degrees) and the Euclidean distance between UE m and UAV n , respectively. In particular, the probability of non-line-of-sight (NLoS) transmission between UE m and UAV n can be obtained as $P_{m,n}^{NLoS} = 1 - P_{m,n}^{LoS}$. Following the free space path-loss (PL) model, the channel's power gain between UE m and UAV n at time slot t is given by

$$h_{m,n}^t = \omega_0^{-1} (d_{m,n}^{elev,t})^{-\alpha} [P_{m,n}^{LoS} \mu_{LoS} + P_{m,n}^{NLoS} \mu_{NLoS}]^{-1}, \quad (8)$$

where $\omega_0 = (\frac{4\pi f_c}{c})^2$, and f_c represents the carrier frequency and c denotes the speed of light. We denote α as the path loss exponent. The average additional losses for LoS and NLoS connections, denoted by μ_{LoS} and μ_{NLoS} respectively, are constants determined by the specific propagation environment.

Without loss of generality, we assume that the ground UEs can communicate with their respective serving UAV via orthogonal frequency-division multiple access (OFDMA) [12], [18], [26]. Then, the interference between different UEs within the UAV's coverage area can be eliminated. Therefore, the achievable data transmission rate between the UE m and UAV n is given by

$$\Omega_{m,n}^t = B_0 \log_2 \left[1 + \frac{p_m^t h_{m,n}^t}{B_0 N_0} \right], \forall m \in \mathcal{M}, t \in \mathbb{T}, \quad (9)$$

where B_0 is the allocated channel bandwidth and N_0 denotes the power spectral density of the additive white Gaussian noise (AWGN) at the UAV. The transmit power of UE m is constrained by

$$0 \leq p_m^t \leq p_{\max}, \forall m \in \mathcal{M}, t \in \mathbb{T}, \quad (10)$$

where $p_{\max}$ denotes the UE's maximum transmit power.

In this work, we assume that the size of each job request follows a normal distribution, i.e., $\Omega_k \sim \mathcal{N}(\Omega^{th}, \sigma^2)$, where Ω^{th} represents the threshold for data transmission.

Lemma 1: For a UAV's coverage region with its radius of Z , in order to ensure that any UE within this region can upload its job request to the UAV with a minimum achievable data transmission rate of at least Ω^{th} , the UE's maximum transmit power must satisfy the following lower bound as

$$p_{\max} \geq \omega_0 \mu_{NLoS} B_0 N_0 \left(2^{\frac{\Omega^{th}}{B_0}} - 1 \right) \left| \sqrt{Z^2 + H^2} \right|^\alpha. \quad (11)$$

The detailed proof of Lemma 1 is presented in Appendix A.

Furthermore, we incorporate Rician and Rayleigh fading channels [27] to quantify their effects on UAV-UE association and transmit power, as formulated in Lemma 2.

Lemma 2 (Path-loss-only vs. fading channels): Let Ω^{th} be the per-slot upload demand, $p_{\max}$ the maximum transmit power. Define $C = B_0 N_0 (2^{\Omega^{th}/B_0} - 1)$, the pure PL gain is $h = G(d)$, and the fading gain $h = G(d)Y$ (for unit-mean Rician Y with factor $\kappa > 0$, and Rayleigh with $\kappa = 0$). Thus, for any

$d \in (0, d_{\max}]$, the minimum-gain threshold

$$y := Y_{\min}(d) = \frac{C}{G(d) p_{\max}} = (d/d_{\max})^\alpha \in (0, 1].$$

i) *Success Probability of job request upload*

$$P_{\text{succ}}^{\text{Rayleigh}} < P_{\text{succ}}^{\text{Rician}} < P_{\text{succ}}^{\text{PL}} = 1. \quad (12)$$

ii) *Expected transmit power conditioned on success.*

For $\bar{p}_{\text{PL}} = \frac{C}{G(d)}$, $\bar{p}_{\text{Ric}} = \frac{C}{G(d)} \Phi_{\text{Ric}}(\kappa, y)$, $\bar{p}_{\text{Ray}} = \frac{C}{G(d)} g(y)$, where $\Phi_{\text{Ric}}(\kappa, y) := \mathbb{E}[1/Y|Y \geq y]$, $g(y) := e^y E_1(y)$. Define the unique thresholds

$$y_\star(\kappa) : \Phi_{\text{Ric}}(\kappa, y_\star) = 1, \quad Y_\star : g(Y_\star) = 1,$$

$$y_\times(\kappa) : \Phi_{\text{Ric}}(\kappa, y_\times) = g(y_\times).$$

Thus,

$$\bar{p}_{\text{PL}} < \bar{p}_{\text{Ric}} < \bar{p}_{\text{Ray}}, \quad 0 < d < d_{\text{crit}}, \quad (13)$$

where $d_{\text{crit}} = d_{\max} \min\{Y_\star^{1/\alpha}, y_\star(\kappa)^{1/\alpha}, y_\times(\kappa)^{1/\alpha}\}$.

The detailed proof of Lemma 2 is presented in Appendix B.

C. UAV Enabled Data Processing Model

The UAV, served as a mobile data processing platform, operates in three stages during each time slot: (1) Moving to a new location; (2) Establishing UE-UAV associations and receiving job requests; (3) Performing computation based on its available resource and request queue state.

1) *UAV Movement Model:* At the beginning of time slot t , the UAV flies from its last location l_n^{t-1} to its next location l_n^t with a flight direction ψ_n^t and a flight speed v_n^t . Actually, the variable ψ_n^t represents the flight direction of the UAV in current time slot t , which is updated from its flight direction ψ_n^{t-1} in last time slot $t-1$ by adding the rotation angle φ_n^t to ψ_n^{t-1} . The rotation angle φ_n^t is constrained within a bounded range, i.e., $\varphi_n^t \in [\varphi_{\min}, \varphi_{\max}]$ (e.g., $\varphi_n^t \in [-(\pi/4), (\pi/4)]$). Based on the updated flight direction ψ_n^t , the UAV selects a flight speed constrained by a maximum allowable speed

$$v_n^t = \|l_n^t - l_n^{t-1}\|_2 \leq V_{\max}. \quad (14)$$

Thus, the movement of the UAV is jointly decided by the set $\{l_n^{t-1}, \psi_n^{t-1}, \varphi_n^t, v_n^t\}$, and the location of the UAV evolves as

$$\begin{aligned} x_n^t &= x_n^{t-1} + v_n^t \cos(\psi_n^{t-1} + \varphi_n^t), \\ y_n^t &= y_n^{t-1} + v_n^t \sin(\psi_n^{t-1} + \varphi_n^t). \end{aligned} \quad (15)$$

2) *Job Request Upload Model:* A job request can be uploaded to a certain UAV only when the corresponding UE is located within the UAV's coverage area, which is defined in Inequality (6). In addition, the resource type required by the job must be included in the UAV's onboard resource set, i.e.,

$$r_k \in \mathcal{R}_n, \forall j_k \in \Gamma, n \in \mathcal{N}, \quad (16)$$

where $\mathcal{R}_n$ is the set of resource types deployed on UAV n and the total set of resource types deployed on all UAVs is $\bigcup_{n \in \mathcal{N}} \mathcal{R}_n = \mathcal{R}$. For each UE m , the amount of uploaded data during a time slot is represented as a function of the transmit power and the channel's power gain: $\Omega_{m,n}^t = f(p_{m,n}^t, h_{m,n}^t)$. Since the time is divided into equal-length slots, we assume that both the UAV and UE remain stationary during each time slot and the channel gain keeps constant during each time slot as well. Fig. 3 illustrates the relationship between the uploaded data size and the UE's transmit power $p_{m,n}^t$, under the assumption that the channel gain

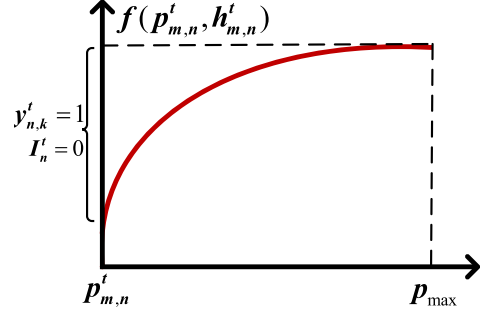


Fig. 3. The uploaded data size is a function of the UE's transmit power and the channel gain between the UE and its associated UAV at each time slot.

$h_{m,n}^t$ remains fixed within a time slot. The channel gain $h_{m,n}^t$ is determined by the current locations of the UE and the UAV. For example, if a certain UE located far from the UAV (but within the UAV's coverage area) attempts to upload a job request with a large data volume, it is likely that the uploading cannot be completed within the current time slot. The binary variable $y_{n,k}^t \in \{0, 1\}$, which indicates the association between UAV n and job j_k of UE m , and the idle state variable I_n^t of UAV n are defined as:

- If the association between the UE and the UAV satisfies Constraints (6) and (16) and $f(p_{m,n}^t, h_{m,n}^t) \geq \Omega_k$, i.e., there exists $p_{m,n}^t \leq p_{\max}$ that enables the successful job upload in the current time slot, then set $y_{n,k}^t = 1$, $I_n^t = 0$.
- Conversely, if $f(p_{m,n}^t, h_{m,n}^t) < \Omega_k$, even when using the maximum transmit power $p_{\max}$, the job cannot be successfully uploaded. Therefore, we set $y_{n,k}^t = 0$, $I_n^t = 1$.
- In another case, if there exist multiple requests $j_k \in \gamma_m^t$ satisfying $y_{n,k}^t = 1$, the UAV remains in a busy state from time t to $t + \sum_{j_k \in \gamma_m^t} y_{n,k}^t$, i.e., $I_n^t = 0, \forall t' \in [t, t + \max\{\sum_{j_k \in \gamma_m^t} y_{n,k}^t, \forall m \in \mathcal{M}\}]$. This indicates that the UAV must wait until all requests from UEs that meet the uploading requirement have been uploaded.

It should be noted that each job can only be uploaded to a single UAV. This also implies that each UE is associated with a single UAV during each time slot. Accordingly, the following association constraint must be satisfied:

$$y_{n,k} \in \{0, 1\}, \quad \sum_{n \in \mathcal{N}} y_{n,k} \leq 1, \forall j_k \in \Gamma. \quad (17)$$

3) *Instance Allocation and Execution Model:* The UAVs, functioning as mobile servers, are pre-configured with computational resources for a set of supported service types $\mathcal{R}_n \subset \mathcal{R}$. The set of resource capacity deployed on UAV n is denoted by $vm_n = \{vcpu_n^r, vmem_n^r\} | r \in \mathcal{R}_n\}$, where $vcpu_n^r$ and $vmem_n^r$ denote the number of virtual CPU cores and the amount of memory configured for service type r , respectively. The UAV allocates computational resources to tasks in its job queue $\mathcal{D}_n^t$ based on its available resource capacity vm_n to execute the corresponding jobs. Specifically, it determines the number of execution instances and allocates the necessary resources for each instance. All execution instances assigned to a task have identical CPU cores and memory requirements. The number of computation instances assigned to execute task v_k^i in vm_n^r at

each time slot t is denoted by $x_{v_k^i, vm_n^r}^t$, where $x_{v_k^i, vm_n^r}^t \in \mathbb{Z}_0^+$ indicates the actual level of parallelism for executing task v_k^i . The resource constraints for instance execution at each time slot $t \in \mathbb{T}$ are defined as

$$\sum_{v_k^i \in \mathcal{D}_n^t} x_{v_k^i, vm_n^r}^t \cdot vcpu_k^i \leq vcpu_n^r, \forall r \in \mathcal{R}_n, \forall n \in \mathcal{N}, \quad (18)$$

$$\sum_{v_k^i \in \mathcal{D}_n^t} x_{v_k^i, vm_n^r}^t \cdot vmem_k^i \leq vmem_n^r, \forall r \in \mathcal{R}_n, \forall n \in \mathcal{N}. \quad (19)$$

D. Energy Consumption Model

In general, the energy consumption of the UE consists of the movement energy and the energy used for uploading job requests. Since the mobility of the UE is self-controlled in practical scenarios, in this work, we only consider the UE's energy consumption of data uploading, which is defined as

$$E_m^t = \sum_{j_k \in \gamma_m^t} y_{n,k}^t \cdot p_m^t, \quad \forall m \in \mathcal{M}, \forall t \in \mathbb{T}. \quad (20)$$

In general, the UAV's energy consumption consists of the energy used during flight, hovering, communication, and computation. Since the energy consumed during either communication or computation is negligible compared to the flight (hovering) energy consumption [6], [28], in this work, we only consider the UAV's flight and hovering energy consumption. According to [29], we assume that the UAV's flight energy consumed during each time slot, i.e., the flight power, is defined as

$$p_n^t(v_n^t) = P_0 \left(1 + \frac{3(v_n^t)^2}{U_{\text{tip}}^2} \right) + P_i \left(\sqrt{1 + \frac{(v_n^t)^4}{4v_0^4}} - \frac{(v_n^t)^2}{2v_0^2} \right)^{\frac{1}{2}} + \frac{d_0 \iota s A (v_n^t)^3}{2}, \quad \forall n \in \mathcal{N}, \forall t \in \mathbb{T}, \quad (21)$$

where P_0 , P_i , U_{tip} , v_0 , d_0 , s , ρ , and A represent the UAV's constant mechanical parameters. The total flight energy consumption of the n -th UAV during time slot t is expressed as

$$E_n^t = p_n^t(v_n^t), \forall n \in \mathcal{N}, \forall t \in \mathbb{T}. \quad (22)$$

In particular, we use $p_n^t(v_n^t = 0)$ to represent the UAV's hovering energy consumption during time slot t [29].

IV. PROBLEM FORMULATION

We define a missed job as one that contains at least one sub-task which fails to complete its computation before its deadline, i.e., violating Constraint (3). Our objective is to minimize the job miss rate and the average energy consumption of both UAVs and UEs within a finite time horizon T , subject to each UAV's resource and UE association constraints. To eliminate scale differences among performance metrics, we normalize them using the maximum transmit power of UEs and the maximum propulsion power of UAVs as reference values. The problem is formulated as follows.

$$\min_{n \in \mathcal{N}, j_k \in \Gamma, r \in \mathcal{R}} F(\mathcal{Q}) = \frac{1}{T} \sum_t \left\{ \left(\frac{N_{\text{miss}}^t}{N_{\text{succ}}^t + N_{\text{miss}}^t} \right) + \zeta \right.$$

$$\left. \left(\frac{1}{\sum_{j_k \in \Gamma} y_{n,k}^t} \frac{\sum_{j_k \in \Gamma} E_m^t}{\sum_{j_k \in \Gamma} y_{n,k}^t p_{\text{max}}} + \frac{1}{N} \frac{\sum_{n=1}^N E_n^t}{\sum_{n=1}^N p_n^t(v_{\text{max}})} \right) \right\} \quad (23)$$

subject to

$$(6), (14), (17), (18), (19),$$

$$0 \leq p_m^t \leq p_{\text{max}}, \forall m \in \mathcal{M}, t \in \mathbb{T}, \quad (24)$$

$$x_{v_k^i, vm_n^r}^t \in \mathbb{Z}_0^+, \forall v_k^i \in \mathcal{D}_n^t, r \in \mathcal{R}_n, \forall n \in \mathcal{N}, \forall t \in \mathbb{T}, \quad (25)$$

where $\mathcal{Q} = \{l_n^t, p_n^t, y_{n,k}^t, x_{v_k^i, vm_n^r}^t\}$ is the variable set and ζ is the weight used to balance the job miss rate and energy consumption. Constraint (6) specifies the communication coverage of the UAV, and (14) defines the maximum flight speed of each UAV. Constraints (18) and (19) ensure that the total CPU cores and memory resources consumed by all executed instances on each UAV during a time slot do not exceed the UAV's resource capacity. Constraint (17) ensures that each job request can be uploaded to at most one UAV. Constraint (24) specifies the maximum transmit power of each UE. Constraint (25) ensures that the number of computation instances allocated to each sub-task is a non-negative integer.

Each UAV aims to appropriately plan its flight trajectory to receive more job requests while reducing the energy consumption of both itself and the associated UEs. Meanwhile, each UAV needs to navigate carefully to avoid collisions with either obstacles or other UAVs, due to the lack of sufficient communication among UAVs. We assume that each UAV has a physical size of radius r_d , and all potential obstacles are modeled with the same physical size of radius r_d . In each time slot t , a safety violation occurs if the Euclidean distance between the center of UAV n and the center of obstacle (another UAV) o_j is smaller than $2r_d$, i.e., $\|l_n^t - o_j^t\|_2 < 2r_d$. We define C_n^t as the safety violation cost of UAV n at time slot t , which is given as follow [30].

$$C_n^t = \begin{cases} 2r_d - \|l_n^t - o_j^t\|_2, & \text{if } \|l_n^t - o_j^t\|_2 < 2r_d; \\ 0, & \text{otherwise.} \end{cases} \quad (26)$$

It can be observed that the smaller distance between the UAV and the obstacle results in the severer safety violation. Therefore, in this work, we aim to minimize the cumulative safety violation costs during UAV operation, while reducing the effect on the overall optimization objective $F(\mathcal{Q})$.

The problem is a complex multi-objective optimization problem involving multi-UAV cooperative trajectory planning. To effectively capture the sequential and stochastic nature of UAV decision-making, the discrete-time trajectory planning is naturally formulated as a Markov decision process (MDP), in which each UAV selects a flight decision based on its current state, resulting in a probability distribution over future states. Reinforcement learning (RL) is well-suited for solving such MDPs by learning a policy that maximizes the expected cumulative reward, while naturally accommodating multi-objective optimization problems without requiring explicit convexity assumptions, and effectively balancing competing objectives through reward design and policy exploration.

V. PROPOSED METHOD

In the following, we first model our problem as a decentralized partially observable Markov decision process (Dec-POMDP), which generalizes the single-agent POMDP to cooperative multi-agent setting [31]. Then, we introduce the multi-agent safe reinforcement learning algorithm framework. Finally, we illustrate the design and implementation details of the proposed SSAC-MGI algorithm.

A. Dec-POMDP Formulation

The Dec-POMDP effectively models a team of cooperative agents operating in a stochastic and partially observable environment. It is formally defined as a tuple $\mathcal{G} = \langle \mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{O}, \mathcal{R}, \gamma \rangle$, where $\mathcal{N}$ is the set of N UAV agents, and $\mathcal{S}$ is a set of states. The sets of joint actions and joint observations are denoted as $\mathcal{A} \equiv \{\mathcal{A}_n\}_{n \in \mathcal{N}}$ and $\mathcal{O} \equiv \{\mathcal{O}_n\}_{n \in \mathcal{N}}$, respectively. The state transition probability space is denoted by $\mathcal{P}$, and $\mathcal{R}$ denotes the reward function. The discount factor $\gamma \in [0, 1]$ denotes the temporal discounting of future rewards.

Observation space $\mathcal{O}$: $\mathcal{O}_n$ denotes the set of observations available to UAV n . At each time slot t , the environment emits a joint observation $o^t = \{o_1^t, \dots, o_N^t\}$, where each UAV n only has access to its local observation $o_n^t \in \mathcal{O}_n$. The local observation o_n^t encapsulates the relevant state information required for UAV n to make movement decisions, which includes the UAV's own status information and the task-related information in its job queue $\mathcal{D}_n^t$. Specifically, the status information includes the current idle state $\mathbf{I}_n^t$, 3D position l_n^t , movement orientation φ_n^t , and the available type-specific onboard resources of UAV n . Moreover, the resource utilization reflects the proportion of resources consumed by the execution of task instances within the job queue. Specifically, the CPU utilization is defined as

$$u_{vcpu_n}^t = \frac{\sum_{j_k \in \mathcal{D}_n^t} \sum_{v_k^i \in j_k} vcpu_k^i \cdot x_{v_k^i, vmr_n}^t}{vcpu_n^r}, \quad (27)$$

and the memory utilization is defined as

$$u_{vmem_n}^t = \frac{\sum_{j_k \in \mathcal{D}_n^t} \sum_{v_k^i \in j_k} vmem_k^i \cdot x_{v_k^i, vmr_n}^t}{vmem_n^r}. \quad (28)$$

Therefore, the local observation is defined as

$$o_n^t = \{\mathbf{I}_n^t, l_n^t, \varphi_n^t, \langle u_{vcpu_n}^t, u_{vmem_n}^t \rangle_{r \in \mathcal{R}_n}\} \in \mathcal{O}_n. \quad (29)$$

The concatenated local observations from all UAVs, denoted as $o^t = \{o_n^t\}_{n \in \mathcal{N}}$, is defined as the global state $s^t \in \mathcal{S}$.

Action space $\mathcal{A}$: $\mathcal{A}_n$ denotes the set of admissible actions for UAV n , which remains consistent across all UAVs in terms of the action dimensionality. At each time step t , each UAV n selects a flight action a_n^t , forming the joint action $a^t = \{a_1^t, \dots, a_N^t\}$. The individual action a_n^t represents the UAV's movement decision, which consists of a rotation angle $\varphi_n^t \in [\varphi_{\min}, \varphi_{\max}]$ and a flight speed $v_n^t \in [0, V_{\max}]$. Accordingly, the action taken by the UAV n at time slot t is represented as

$$a_n^t = \{\varphi_n^t, v_n^t\} \in \mathcal{A}_n. \quad (30)$$

In particular, for UAVs in busy states (i.e., $\mathbf{I}_n^t = 0$), the action is fixed as $a_n^t = \{\varphi_n^{t-1}, 0\}$, indicating that the UAV remains stationary while maintaining its previous orientation.

State transition probability $\mathcal{P}$: The impact of a joint action on the environment is characterized by the transition probability function $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$, which defines the state transition dynamics through the probability distribution $P(s^{t+1}|s^t, a^t)$. It specifies the likelihood of transitioning from the current state s^t to the next state s^{t+1} upon executing the joint action a^t , which corresponds to the movement decisions of all idle UAVs. In this work, the transition is deterministic, as the next state is fully determined by the current state and the joint action of all idle UAVs.

Reward function $\mathcal{R}$: At each transition, the environment emits a reward defined by the mapping $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{R}$, which is designed to minimize the job miss rate and the average energy consumption of both UAVs and UEs. As the two objectives specified in Equation (23) are directly influenced by the UAV trajectories, the reward function is accordingly defined as Equation (31). The reward function is designed to guide the policy towards learning UAV trajectories that prioritize the resource-matched job requests, thereby minimizing both the job miss rate and the energy consumption during flight.

$$R_n^t(o_n^t, a_n^t) = \begin{cases} \frac{N_s^t}{N_s^t + N_n^t}, & \text{if } \sum_{j_k \in \Gamma} y_{n,k}^t > 0, \\ \frac{N_s^t}{N_s^t + N_n^t} - \zeta \left(\frac{\sum_{j_k \in \Gamma} E_m^t}{\sum_{j_k \in \Gamma} y_{n,k}^t p_{\max}} + \frac{E_n^t}{p_n^t(v_{\max})} \right), & \text{otherwise} \end{cases} \quad (31)$$

where N_s^t denotes the number of tasks completed before their deadlines, and N_n^t denotes the number of tasks that miss their deadlines. Based on the reward function, if a UAV accepts at least one request, its reward is determined by the task success rate. Otherwise, the reward balances the task success rate and the energy consumption on both the UAV and UE sides.

B. Multi-Agent Safe Reinforcement Learning Framework Based on Markov Games of Intervention

Reinforcement learning enables autonomous agents to learn complex behaviors through the interaction with the environment to maximize expected returns. However, using the trial-and-error exploration during the training process may lead to unsafe actions and cause certain damages to the agent. In this work, UAVs are modeled as agents that are trained under the multi-agent reinforcement learning (MARL) framework to learn movement strategies that optimize system-level objectives. However, for pursuing high-reward regions, UAVs may risk colliding with obstacles including other UAVs. To address the challenge of safe exploration, prior work has adopted either constrained Markov decision process (c-MDP) formulations [32] or reward shaping techniques [33] to guide policies towards constraint satisfaction. However, c-MDP based methods are usually difficult to be extended to settings with unknown dynamics, while reward shaping typically provides no safety guarantees during the learning process. To enable finer-grained control, we propose a two-agent Markov game model for each UAV, in which a *Standard Agent* learns a reward-maximizing policy, while a *Safety Agent* enforces system safety both during and after training.

Two-Agent based Dec-POMDP: To support our proposed two-agent model, we extend the standard Dec-POMDP formulation in Subsection V-A to $\mathcal{W} = \langle \{g_1^{\text{stand}}, g_2^{\text{safe}}\}, \mathcal{S}, \mathcal{A}, \mathcal{A}^{\text{safe}}, \mathcal{P}, R_1, R_2, \gamma \rangle$, where $\mathcal{A}^{\text{safe}} \subseteq \mathcal{A}$

denotes the action set available to the *Safety Agent* g_n^{safe} . We define two reward functions: $R_1 : \mathcal{S} \times \mathcal{A} \times \mathcal{A}^{\text{safe}} \rightarrow \mathbb{R}$ for the *Standard Agent* g_n^{stand} , and $R_2 : \mathcal{S} \times \mathcal{A} \times \mathcal{A}^{\text{safe}} \rightarrow \mathbb{R}$ for the *Safety Agent* g_n^{safe} . The transition probability function $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{A}^{\text{safe}} \rightarrow [0, 1]$ takes the states and actions of both agents as input. The *Standard Agent* and the *Safety Agent* operate under Markov policies $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ and $\pi^{\text{safe}} : \mathcal{S} \times \mathcal{A}^{\text{safe}} \rightarrow [0, 1]$, respectively, drawn from the policy sets Π and $\Pi^{\text{safe}} \subset \Pi$. Notably, Π^{safe} consists of deterministic mappings, whereas Π comprises stochastic policies. This distinction allows the *Safety Agent* to override the *Standard Agent*'s exploratory actions and to execute precise, risk-averse behaviors to prevent the system from entering unsafe states. The action overriding is implemented through a triggered intervention, which is determined by a binary policy $\mathbf{g} : \mathcal{S} \rightarrow 0, 1$ integrated into the *Safety Agent*.

Standard Agent Objective: The *Standard Agent* aims to learn an optimal policy $\pi^* \in \Pi$ that maximizes the expected cumulative reward. In our context, this objective corresponds to minimizing the job miss rate and reducing the average energy consumption of both UAVs and UEs throughout the training process. At each time step, the *Standard Agent* samples an action $a_t \sim \pi(\cdot | s_t)$, while the *Safety Agent* selects an action $a_t^{\text{safe}} \sim \pi^{\text{safe}}(\cdot | s_t)$ when an intervention is triggered, which is determined by the binary intervention policy $\mathbf{g}(s_t) \in \{0, 1\}$. The action actually executed by the UAV is defined as

$$\tilde{a}_t = \mathbf{g}(s_t) \cdot a_t^{\text{safe}} + (1 - \mathbf{g}(s_t)) \cdot a_t, \quad (32)$$

where the *Safety Agent* overrides the *Standard Agent* if and only if an intervention occurs (i.e., $\mathbf{g}(s_t) = 1$). The *Standard Agent* follows its stochastic policy in states where no intervention occurs, aiming to find an optimal policy that maximizes the expected cumulative reward under the influence of the *Safety Agent* as follow:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi, (\pi^{\text{safe}}, \mathbf{g})} \left[\sum_{t=0}^T \gamma^t R_1^t(s_t, \tilde{a}_t) \middle| s_0 = s \right]. \quad (33)$$

The corresponding reward function for the *Standard Agent* at each time step is given by

$$R_1^t(s_t, \tilde{a}_t) = \mathbf{g}(s_t) \cdot R(s_t, a_t^{\text{safe}}) + (1 - \mathbf{g}(s_t)) \cdot R(s_t, a_t), \quad (34)$$

which ensures that the reward is assigned according to the action actually executed and the location where the intervention occurs.

Safety Agent Objective: The *Safety Agent* aims to learn an intervention mechanism comprising a gating policy $\mathbf{g}$ and a safety policy π^{safe} , which together minimize the risk of unsafe actions while avoiding excessive interference with the *Standard Agent*. Similar to the *Standard Agent*, it seeks an optimal policy $(\pi^{\text{safe}}, \mathbf{g})^* \in \Pi^{\text{safe}}$ that maximizes the expected return, which is defined as

$$(\pi^{\text{safe}}, \mathbf{g})^* = \arg \max_{(\pi^{\text{safe}}, \mathbf{g})} \mathbb{E}_{\pi, (\pi^{\text{safe}}, \mathbf{g})} \left[\sum_{t=0}^T \gamma^t R_2^t(s_t, \tilde{a}_t) \middle| s_0 = s \right]. \quad (35)$$

To encourage selective and meaningful interventions, the *Safety Agent* incurs a cost whenever it overrides the *Standard Agent*'s action. The reward function for the *Safety Agent* at each time

Algorithm 1: SSAC-MGI.

Input: The UAV set $\mathcal{N}$, the shared critic NN Q_ω and policy NN π_θ for each SAC module
Output: The optimal cooperative $\pi^*, (\pi^{\text{safe}}, \mathbf{g})^*$

- 1 Initialize data buffers $\mathcal{C}^{\text{stand}}, \mathcal{C}^{\text{safe}}, \mathcal{C}^{\text{int}}$
- 2 Initialize hyper-parameters $\theta, \omega_1, \omega_2$, and target network $\bar{\omega}_1 \leftarrow \omega_1, \bar{\omega}_2 \leftarrow \omega_2$ for SAC module
- 3 **for** $episode \leftarrow 1$ **to** $episode_{\text{max}}$ **do**
- 4 $t \leftarrow 1$
- 5 **while** $t \leq T$ **do**
- 6 **for** $n \in \mathcal{N}_{\text{idle}}^t$ (set of idle UAVs at t) **do**
- 7 Obtain o_n^t and s_n^t
- 8 Sample a standard action $a_{n,t}^{\text{stand}} \sim \pi_\theta(\cdot | o_n^t)$, a safe action $a_{n,t}^{\text{safe}} \sim \pi_\theta^{\text{safe}}(\cdot | o_n^t)$, and an intervention action $a_{n,t}^{\text{int}} \sim \mathbf{g}(\cdot | o_n^t) \in \{0, 1\}$
- 9 **if** $a_{n,t}^{\text{int}} = 0$ **then**
- 10 Apply standard action $\tilde{a}_n^t = a_{n,t}^{\text{stand}}$
- 11 **else if** $a_{n,t}^{\text{int}} = 1$ **then**
- 12 Apply safe action $\tilde{a}_n^t = a_{n,t}^{\text{safe}}$
- 13 State transition $o_n^{t+1} \sim P(\cdot | \tilde{a}_n^t, s_n^t)$
- 14 Ensure that $y_{n,k}^t, \forall j_k \in \Gamma$, and allocate instances to $v_k^t \in \mathcal{D}_n^t$ using **Algorithm 2**
- 15 Compute reward $R(o_n^t, \tilde{a}_n^t)$ and safety violation $C(o_n^t, \tilde{a}_n^t)$ for all $n \in \mathcal{N}_{\text{idle}}^t$
- 16 Obtain $R_1^t, R_2^t, R_{\text{int}}^t$ for all $n \in \mathcal{N}_{\text{idle}}^t$
- 17 Add the samples $(o_n^t, \tilde{a}_n^t, R_1^t, o_n^{t+1}), (o_n^t, \tilde{a}_n^t, R_2^t, o_n^{t+1})$, and $(o_n^t, a_{n,t}^{\text{int}}, R_{\text{int}}^t, o_n^{t+1})$ of all UAVs to $\mathcal{C}^{\text{task}}, \mathcal{C}^{\text{safe}}, \mathcal{C}^{\text{int}}$, respectively.
- 18 $t \leftarrow t + 1$
- 19 **for** Each policy in $\{\pi, (\pi^{\text{safe}}, \mathbf{g})\}$ **do**
- 20 Sample a batch of $\mathcal{B}$ from its corresponding $\mathcal{C}$
- 21 Update $\omega_i \leftarrow \omega_i - \lambda_Q \widehat{\nabla}_{\omega_i} J_Q(\omega_i)$ for $i \in \{1, 2\}$
- 22 Update $\theta \leftarrow \theta - \lambda_\pi \widehat{\nabla}_\theta J_\pi(\theta)$
- 23 Update $\eta \leftarrow \eta - \lambda \widehat{\nabla}_\eta J(\eta)$ [34]
- 24 Update $\bar{\omega}_i \leftarrow \tau \omega_i + (1 - \tau) \bar{\omega}_i$, for $i \in \{1, 2\}$

step is defined as:

$$R_2^t(s_t, \tilde{a}_t) = - \left[\mathbf{g}(s_t) \cdot C(s_t, a_t^{\text{safe}}) + (1 - \mathbf{g}(s_t)) \cdot C(s_t, a_t) \right] - \delta, \quad (36)$$

where $C(\cdot)$ is the cost function that quantifies violations of predefined safety constraints, such as minimum safe distances between UAVs and obstacles (as defined in Equation (26)), and $\delta \in \mathbb{R}^+$ is a fixed penalty associated with each intervention. The intervention policy $\mathbf{g}(s_t)$ is optimized using the same reward function structure, denoted by

$$R_{\text{int}}^t(s_t, \tilde{a}_t) = - \left[\mathbf{g}(s_t) \cdot C(s_t, a_t^{\text{safe}}) + (1 - \mathbf{g}(s_t)) \cdot C(s_t, a_t) \right] - \delta \cdot \mathbf{g}(s_t), \quad (37)$$

which is jointly trained alongside the safety action policy π^{safe} , ensuring coherent behavior and consistent learning objectives within a unified safety-aware framework.

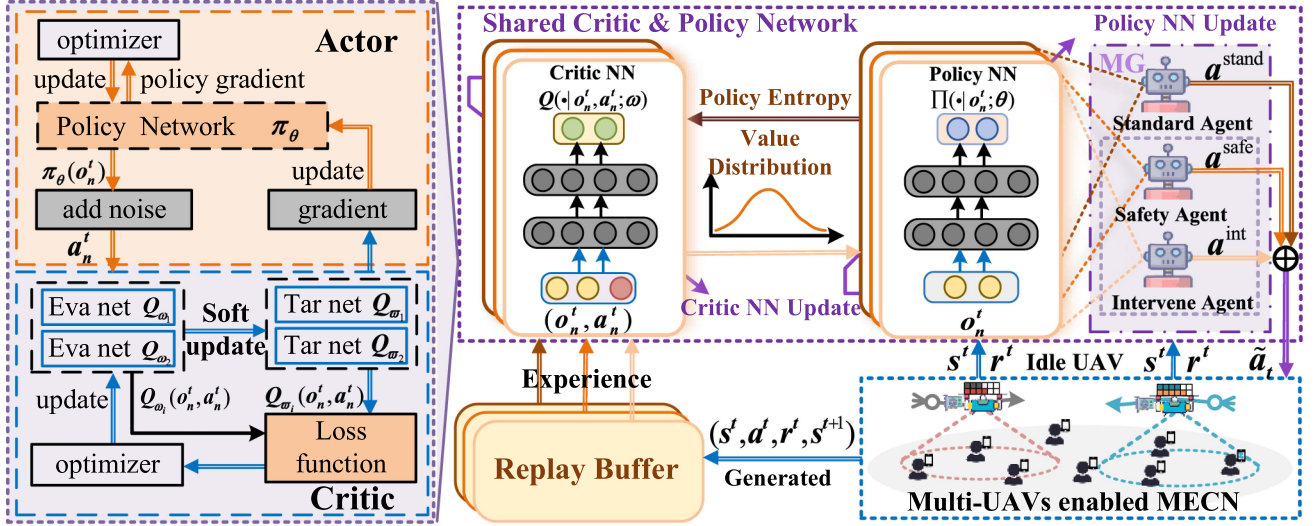


Fig. 4. Algorithm framework.

Safety Intervention in Two-Agent Markov Game: This process refers to a sequential decision-making paradigm involving two agents that alternately control the UAV to balance reward optimization and safety enforcement. The *Safety Agent* learns a set of intervention points, which are characterized by when and where the intervention is needed, to identify critical states and determine appropriate actions to ensure system safety. This intervention mechanism is an integral part of the *Safety Agent's* decision-making process. However, jointly optimizing the intervention policy and the value functions of the safety policy may lead to instability during the training process. To mitigate this issue, we decouple the optimization process by training the intervention policy g separately using a gradient-based RL algorithm named Soft Actor-Critic (SAC) [34]. Building upon the safety intervention mechanism, we design a parallel SAC training architecture comprising both a standard policy and a safety policy (which includes a parallel intervention policy), shared across multiple UAV agents. This architecture enables the UAVs to collaboratively learn trajectory planning policies that are both safe and efficient.

C. Design and Implementation of SSAC-MGI Algorithm

Algorithm Architecture: We design three independent SAC modules to implement the shared network in our proposed multi-agent safe reinforcement learning framework. As illustrated in Fig. 4, these modules correspond to the standard, safety, and intervention policies, which are jointly optimized within the SSAC-MGI framework. Each UAV interacts with the MECN environment via the shared network integrating these three policies. The interaction data are stored separately in policy-specific replay buffers, each of which is used to train and update its corresponding SAC module. The entire system operates as specified by Algorithm 1. Specifically, we begin by initializing the replay buffers for the three SAC models, along with their actor (i.e., policy) parameters θ and critic network parameters ω_i (for $i \in \{1, 2\}$), as well as the corresponding target

network parameters $\bar{\omega}_i$. At each timestep, idle UAVs forward their local observations to the three shared policies. Guided by $g(o_n^t)$, each UAV performs $\hat{a}_n^t$ from the standard or safety policy, triggering the joint state transition. This action schedules UAV trajectories, thereby determining the spatial-temporal placement of tasks across the fleet. Each UAV subsequently invokes the inner round-robin (RR) module (Algorithm 2) to allocate UE-requested resources based on its instantaneous load. The reward from the joint effect of UAV trajectory and resource allocation establishes a feedback loop that iteratively updates the UAV scheduling policies. Line 17 of Algorithm 1 routes interaction data to dedicated replay buffers for subsequent SAC updates. Thus, RR serves as an embedded module within the policy learning framework, enabling trajectory-planning-driven joint optimization of scheduling and resource allocation.

Round-Robin Based Instance Allocation: We propose a Round-Robin strategy to allocate task instances requested by job workflows to virtual machines (VMs) deployed on UAVs. At the beginning of each time slot, the available CPU cores and memory resources of all VMs are reset. UEs upload their job requests to the UAVs based on their current locations, as described in Step (2) of Subsection III-C. The accumulated job requests $\mathcal{D}_n^t$ are then sorted based on the remaining execution time of the most urgent task in each job. Each UAV iteratively selects jobs from the sorted list and prioritizes tasks within each job by the task's deadline urgency. For each task, the UAV first checks whether all instances required for execution ($\phi_k^i \cdot num_k^i$) have been completed. If not, it calculates the maximum number of instances that can be allocated based on the currently available CPU and memory resources. The actual allocation is further adjusted depending on whether the task is approaching its deadline. Specifically, deadline-imminent tasks are prioritized and allocated sufficient execution instances to ensure timely completion, while those with sufficient remaining time are allocated fewer execution instances, based on the average number of execution instances per remaining available time slot, thereby ensuring that as many tasks as possible are executed promptly and do not

Algorithm 2: Round-Robin for Instance Allocation.

```

1 Reinitialize  $vcpu_n^r$  and  $vmem_n^r \forall r \in \mathcal{R}_n, n \in \mathcal{N}$ 
2 for  $n \in \mathcal{N}$  do
3   Update  $\mathcal{D}_n^t$  after job requests are uploaded by UEs (see
   Step (2) in Subsection III-C)
4   Sort  $\mathcal{D}_n^t(\uparrow) = \{j_k \in \mathcal{D}_n^t \mid \min_{v_k^i \in j_k} (\xi \phi_k^i - (t - t_k^g))\}$ 
5   Initialize  $Rr \leftarrow 0$  // Round-robin index
6   while  $\mathcal{D}_n^t \neq \emptyset$  and  $Check(vcpu_n^r, vmem_n^r) = \text{True}$  do
7     Sort  $j_k(\uparrow) = \{v_k^i \in j_k \mid \text{sort by } (\xi \phi_k^i - (t - t_k^g))\}$ 
8     for  $v_k^i \in j_k$  do
9       if  $CheckStatus(v_k^i) = \text{done}$  then
10        continue // skip completed task
11         $x'_{v_k^i, vm_n^r} = \min \left\{ \left\lfloor \frac{vcpu_n^r}{vcpu_k^i}, \frac{vmem_n^r}{vmem_k^i} \right\rfloor \right\}$ 
12        if  $\xi \phi_k^i - AWT(v_k^i) \geq \phi_k^i$  then
13           $x_{v_k^i, vm_n^r} = \min \{x'_{v_k^i, vm_n^r}, num_k^i\}$ 
14        else
15           $x''_{v_k^i, vm_n^r} = \frac{num_k^i \cdot \phi_k^i}{\xi \phi_k^i - AWT(v_k^i)}$ 
16          if  $Rr = 0$  and  $x'_{v_k^i, vm_n^r} < x''_{v_k^i, vm_n^r}$  then
17            Remove  $j_k$  from  $\mathcal{D}_n^t$ 
18            Break //  $j_k$  is missed
19          else
20             $x_{v_k^i, vm_n^r} = \min \{x'_{v_k^i, vm_n^r}, x''_{v_k^i, vm_n^r}\}$ 
21           $vcpu_n^r \leftarrow vcpu_n^r - vcpu_k^i \cdot x_{v_k^i, vm_n^r}$ 
22           $vmem_n^r \leftarrow vmem_n^r - vmem_k^i \cdot x_{v_k^i, vm_n^r}$ 
23         $Rr \leftarrow Rr + 1$ 
24        if  $CheckStatus(j_k) = \text{done}$  then
25          Remove  $j_k$  from  $\mathcal{D}_n^t$  // Complete  $j_k$ 

```

experience excessive delays. If a job cannot be satisfied with its required resource during the first round, it is dropped to avoid resource wastage. After each round, the resource counters are updated, and the round-robin index is incremented to ensure fair allocation across all jobs. Additionally, completed jobs are identified and removed from the job queue to prevent redundant instance allocation in subsequent rounds.

Policy Update: In MARL, agents either learn with centralized value functions or rely solely on local observations. MASAC (Multi-Agent SAC) employs a centralized value function to leverage global information, whereas ISAC (Independent SAC) utilizes local observations for both policy and value functions, enabling scalable decentralized learning. Following the ISAC paradigm, our framework treats each UAV as an independent agent with access to its local observations only. To improve the training efficiency, we adopt a shared policy architecture, in which all UAVs share network parameters but interact with the environment and learn independently. Within this structure, the shared policy is optimized to maximize each agent's expected long-term return. To further promote exploration and policy robustness, we adopt an entropy-augmented objective that incorporates a policy entropy term $\mathcal{H}$ into the reward formulation [34], [35]. This encourages agents to explore diverse actions and prevents premature convergence. The objective of each policy

is defined as follow [36].

$$\mathcal{J}_\pi = \mathbb{E}_{(s_t, a_t) \sim \Pi} \left[\sum_{t=0}^T \left[R_t(s_t, a_t) + \eta \mathcal{H}(\pi(\cdot | s_t)) \right] \right], \quad (38)$$

where

$$\begin{aligned} \mathcal{H}(\pi(\cdot | s)) &= - \int_{a \in \mathcal{A}} \pi(a|s) \log \pi(a|s) da \\ &= \mathbb{E}_{a \sim \pi(\cdot | s)} [-\log \pi(a|s)]. \end{aligned} \quad (39)$$

The temperature parameter η controls the trade-off between the entropy term and the reward. As $\eta \rightarrow 0$, maximum entropy reinforcement learning asymptotically converges to conventional reinforcement learning.

To find the optimal policy that maximizes the objective in Equation (39), we evaluate the Q-function of the current policy and update the policy using an off-policy gradient step based on a soft policy iteration formulation. Specifically, in the policy evaluation step of soft policy iteration, we start from any function $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ and repeatedly apply a modified Bellman backup operator $\mathcal{T}^\pi$ defined as

$$\mathcal{T}^\pi Q(s_t, a_t) \triangleq R_t(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V(s_{t+1})], \quad (40)$$

where $V(s_t) = \mathbb{E}_{a_t \sim \pi} [Q(s_t, a_t) - \eta \log \pi(a_t | s_t)]$ is the soft state value function. In the policy improvement step, we update the policy toward the exponential of the new soft Q-function, for each state, we update the policy according to

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot | s_t) \left\| \frac{\exp \left(\frac{1}{\eta} Q^{\pi_{\text{old}}}(s_t, \cdot) \right)}{Z^{\pi_{\text{old}}}(s_t)} \right\| \right), \quad (41)$$

where the partition function $Z^{\pi_{\text{old}}}(s_t)$ normalizes the distribution and can be omitted during optimization [34]. We employ function approximators (i.e., neural networks) for both the soft Q-function and the policy, and alternate between optimizing these networks via stochastic gradient descent (SGD). As described in Algorithm 1, each SAC module employs a tractable policy network $\pi_\theta(\cdot | s_t)$, along with two soft Q-functions modeled by neural networks, parameterized as $Q_{\omega_1}(s_t, a_t)$, $Q_{\omega_2}(s_t, a_t)$. To stabilize training and mitigate Q-value overestimating, each Q-network Q_{ω_i} is accompanied by a target Q-network $Q_{\bar{\omega}_i}$, where $i \in \{1, 2\}$. The soft Q-functions are trained by minimizing the soft Bellman residual:

$$\begin{aligned} J_Q(\omega_i) &= \mathbb{E}_{(s_t, a_t) \sim \mathcal{C}} \left[\frac{1}{2} \left(Q_{\omega_i}(s_t, a_t) - \left(R_t(s_t, a_t) \right. \right. \right. \\ &\quad \left. \left. \left. + \gamma \mathbb{E}_{s_{t+1} \sim p} [V_{\bar{\omega}}(s_{t+1})] \right) \right)^2 \right], \end{aligned} \quad (42)$$

where

$V_{\bar{\omega}}(s_{t+1}) = \mathbb{E}_{a_{t+1} \sim \pi_\theta} [\min_{i=1,2} Q_{\bar{\omega}_i}(s_{t+1}, a_{t+1}) - \eta \log \pi_\theta(a_{t+1} | s_{t+1})]$. The target Q-function parameters are updated using an exponential moving average of the corresponding soft Q-function weights. The policy parameters are updated by minimizing the KL-divergence in Equation (43), scaled by the temperature η and ignoring the constant log-partition function.

$$J_\pi(\theta) = D_{\text{KL}} \left(\pi_\theta^{\text{new}}(\cdot | s_t) \left\| \frac{\exp \left(\frac{1}{\eta} Q^{\pi_{\text{old}}}(s_t, \cdot) \right)}{Z^{\pi_{\text{old}}}(s_t)} \right\| \right)$$

$$= \mathbb{E}_{\mathcal{C}, \pi_\theta} \left[\eta \log \pi_\theta(a_t | s_t) - \min_{i=1,2} Q_{\omega_i}(s_t, a_t) \right]. \quad (43)$$

D. Convergence and Complexity Analysis

Convergence Analysis: Given the safety policy $\pi^{\text{safe}, \mathbf{g}}$, let $\tau_k \in \mathbb{T}$ denote the k -th intervention time. The intervention operator is defined in Equation (44) shown at the bottom of this page. Following the Bellman update rule, the safety Bellman operator used for value iteration is defined in Equation (45) shown at the bottom of this page; it captures the nested, sequential interaction between the two agents and returns the smaller expected future safety cost. The optimal value function $v_{\text{safe}}^*(s)$ is characterized as the unique fixed point of $\mathcal{T}$, which is approximated via the Q-learning variant in Equation (46) (shown at the bottom of this page) with step-size α_t . The stochastic approximation theory ensures almost-sure convergence of this variant to the fixed point, as established by Theorem 1 (optimality) and Theorem 2 and 3 (convergence) in [37], while the optimal value function induces a deterministic sequence of interventions times. Furthermore, by extracting only dimension-invariant features across all UAVs while preserving the linear function-approximation conditions for Q-learning convergence, the shared network enables *Algorithm 1* to inherit above convergence guarantees.

Time Complexity: The SSAC-MGI algorithm jointly trains three parallel SAC modules (standard, safety, and intervention) and incorporates a nested round-robin instance allocator. Let J denote the maximum number of pending jobs per UAV, V the maximum number of sub-tasks per job, d and a the dimensions of the state and action spaces, and B the batch size used for updates. At each time step, the algorithm performs the following operations: (i) *Action & transition:* each idle UAV samples actions from the three policies and updates its state, $O(N)$. (ii) *Instance allocation:* Algorithm 2 runs once per timestep, looping over all N UAVs. For each UAV n , the job queue $\mathcal{D}_t^n (\leq J)$ is first sorted by deadlines in $O(J \log J)$, and re-orders at most V sub-tasks per job incurs $O(V)$ time. Consequently, the step time complexity is $O(NJV \log V)$. (iii) *Reward & buffer update:* each UAV computes its reward and safety cost incurs $O(N)$ time. (iv) *Policy update:* At each episode's end, the three SAC modules are trained on a batch of B samples, their forward and backward propagation require $O(B(d+a))$ time per module. Let E denote the number of training episodes, the overall time complexity is therefore $O(E \cdot (T \cdot NJV \log V + B(d+a)))$.

Space Complexity: the primary sources of memory consumption include the neural network parameters, replay buffers,

instance-allocation storage, and temporary UAV-state storage. Each SAC module maintains an actor, two critic and two target critic networks, let P be the total number of trainable parameters, yielding a memory cost of $O(P)$. Furthermore, three replay buffers (one per SAC module), each storing up to C transition tuples of dimension $O(d+a)$, for $O(C(d+a))$ space overall. To hold all pending sub-tasks across N UAVs, each with up to J jobs of at most of V sub-tasks, requiring $O(NJV)$. In addition, the storage of intermediate states across UAVs and time steps further contributes $O(TNd)$ space. Therefore, the total space complexity is $O(P + C(d+a) + NJV + TNd)$.

VI. EVALUATION

A. Experimental Setup

We implement a simulation environment for the multi-UAV enabled MECN using the proposed SSAC-MGI algorithm, which is built upon the OpenAI Gym interface. To reflect realistic workloads, we incorporate two real-world datasets: the Alibaba Cluster Traces V2017 and V2018 [24]. From these datasets, we filter workflows with job types in $\{3, 8, 10, 12\}$ and task-level parallelism $num_k^i \in [10, 100]$ to simulate job requests from UEs. For static and mobile UEs, we use movement traces from 20 Twitter users located near Oxford Street, London, on March 14, 2018 [38], selecting users who posted at least three tweets that day [10]. Each UAV, acting as a MEC platform, is equipped with two resource types selected from the following configurations: $\{3:(1856, 20.01)\}$, $\{8:(3520, 37.95)\}$, $\{10:(2048, 22.08)\}$, $\{12:(3584, 38.64)\}$, where each VM has 64 CPU cores and 0.690 memory units [39]. In each tuple, the key indicates the resource type, and the value indicates the pre-allocated resources (CPU, memory). Simulation parameters follow [29], [40], with defaults listed in Table III unless otherwise specified. The source code of our algorithm is available on GitHub².

Traces data: We independently filter 1,754 tasks from the 'tbl_batch_task' table in both Trace V2017 and Trace V2018 by selecting those whose durations satisfy $\xi \phi_k^i \in (0, 300]$ for all $v_k^i \in \Gamma$, where $\xi = 1.5$ denotes the UE-specified deadline coefficient [2]. All 4,756,733 instances are associated with 1,754 tasks originating from 912 jobs in Trace V2017, and 2,647,977 instances associated with 1,754 tasks originating from 1,594 jobs in Trace V2018. The job characteristics of the two datasets, including the distributions of job types and requested CPU

²The code of our multi-UAV enabled MECN implementation is publicly available at: <https://github.com/xiulingzhang22/SSAC-MGI>.

$$\mathcal{M}^{(\pi^{\text{safe}}, \mathbf{g})} Q_{\text{safe}}(s_{\tau_k}, a_{\tau_k}^{\text{safe}}) = R_2(s_{\tau_k}, a_{\tau_k}^{\text{safe}}) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s_{\tau_k}, a_{\tau_k}^{\text{safe}}) v_{\text{safe}}^{\pi^{\text{stand}}, (\pi^{\text{safe}}, \mathbf{g})}(s'), \forall \tau_k \in \mathbb{T}. \quad (44)$$

$$\mathcal{T} v_{\text{safe}}^{\pi^{\text{stand}}, (\pi^{\text{safe}}, \mathbf{g})}(s) := \underbrace{\max \left\{ \mathcal{M}^{(\pi^{\text{safe}}, \mathbf{g})} Q_{\text{safe}}(s_{\tau_k}, a_{\tau_k}^{\text{safe}}) \right\}}_{\text{intervention}}, \underbrace{R_{\text{int}}(s, a^{\text{stand}}) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a^{\text{stand}}) v_{\text{safe}}^{\pi^{\text{stand}}, (\pi^{\text{safe}}, \mathbf{g})}(s')}_{\text{no intervention}}. \quad (45)$$

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t) \left[\max \left\{ \mathcal{M}^{(\pi^{\text{safe}}, \mathbf{g})} Q_t(s_t, a_t), R(s_t, a_t) + \gamma \max_{a' \in \mathcal{A}} Q_t(s_{t+1}, a') \right\} - Q_t(s_t, a_t) \right]. \quad (46)$$

TABLE III
KEY PARAMETERS OF SIMULATION

Parameters	Value
Side length of the 2D plane	500×500 (m)
The number of UAVs	$N = \{2, 3, 4, 5, 6\}$
Set of all resource type identifiers	$\mathcal{R} = \{3, 8, 10, 12\}$
The number of UEs	$M = 20$
The azimuth angle of UAV	$\theta = 30^\circ$
The hovering altitude of UAV	$H = 100\text{m}$
The operating period of UAVs	$T = 300\text{s}$
Communication bandwidth of UAV	$B_0 = 1\text{MHz}$
The power spectral density	$N_0 = 1 \times 10^{-20}\text{W/MHz}$
Environment model constants	$\rho = 11.95, \beta = 0.14$
Additional path loss for LoS/NLoS	3dB/23dB
The carrier frequency	$f_c = 2\text{GHz}$
The speed of light	$c = 3 \times 10^8 \text{ m/s}$
Data size of request file (in MB)	$\Omega_k \sim \mathcal{N}(1.7, 0.1)$
The maximum UAV speed	50m/s
Transmit power of the UAVs	$p_n = 5 \text{ W}$
Rotor blade tip speed	$U_{tip} = 120\text{m/s}$
Rotor disc area in m^2	$A = 0.503$
Air density in kg/m^3	$\rho = 1.225\text{kg/m}^3$
Mean rotor induce velocity in hover	$v_0 = 4.03\text{m/s}$
Rotor solidity, Fuselage drag ratio	$s = 0.05, d_0 = 0.6$
Hovering power components	$P_0 = 79.86, P_i = 88.63$
The physical radius of UAV	$r_d = 5\text{m}$
Number of neurons per hidden layer	$h=64$
Reward discount factor	$\gamma=0.99$
Learning rate	$\lambda=0.0001$
The temperature parameter	$\eta=1$
Memory replay buffer	$C=100000$

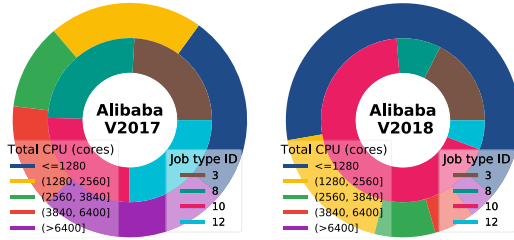


Fig. 5. Job characterization of Alibaba traces. The outer circle shows the number of jobs within each range of required total CPU cores, while the inner circle shows the number of jobs for each resource type category.

cores³, are shown in Fig. 5. At each time slot, UE-generated jobs are randomly sampled from the filtered dataset, following the Poisson arrival rate of $k(\Delta t) = \frac{|\mathcal{R}|}{T}$, as illustrated in Fig. 6(a). The distributions of task duration are shown in Fig. 6(b) and (c). It can be seen that most tasks in Trace V2018 have shorter durations and tighter deadlines.

Baseline algorithms: As no existing methods can directly solve the proposed problem, we compare SSAC-MGI with several adapted baselines and a manual UAV trajectory policy. To ensure a fair comparison, all baselines are implemented within the same system framework as SSAC-MGI.

- **SSAC-MGI First Come First Service (SSAC-MGI-FCFS):** A variant of SSAC-MGI in which the embedded *round-robin* resource allocation strategy is replaced by a classical *first-come-first-served* (FCFS) scheme, where each

³CPU request values in the trace are divided by 100 to reflect the actual request, as the basic unit for batch tasks is 10^{-2} CPU [39].

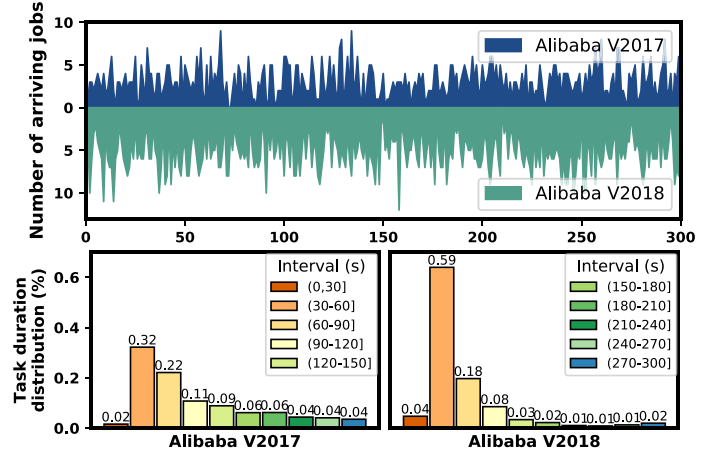


Fig. 6. The Alibaba trace analysis includes: (a) number of arriving job requests per time slot; (b) the duration distribution of 1754 tasks within jobs.

job in the queue is sequentially allocated all the available resources required to execute its tasks.

- **Shared Soft Actor-Critic (SSAC):** An unconstrained MARL baseline where each UAV independently selects actions from a shared SAC policy [34], instead of employing our set of standard, safety, and intervention policies. To minimize the system-level safety cost, safety signals are embedded into the reward function, a typical reward-shaping approach in unconstrained RL. However, such methods often suffer from restricted exploration, potentially leading to suboptimal performance.
- **Shared Trust Region Policy Optimization (STRPO):** An unconstrained MARL baseline similar to SSAC is a reward-shaping RL approach, which is constructed by replacing SAC with Trust Region Policy Optimization (TRPO) [41], while retaining the policy architecture and training procedure used in SSAC-MGI.
- **Shared Constrained Policy Optimization (SCPO):** A constrained MARL baseline based on Constrained Policy Optimization (CPO) [42], which enforces safety constraints by dynamically adjusting a penalty coefficient during training. While integrated into the same shared training framework, CPO differs from our approach in that it directly embeds safety handling into policy optimization. This comparison highlights the benefits of explicitly decoupling safety control from policy learning, particularly in terms of flexibility and performance under complex safety requirements.
- **Manual UAV Trajectory Planning Policy (MANUAL) [18]:** A handcrafted baseline policy that focuses solely on providing computing services. In our implementation, the entire MECN coverage area is partitioned into a $50 \times 50\text{m}$ grid. To ensure full-area coverage and improve service performance, each UAV alternately follows one of two manually designed trajectories with opposite counter-clockwise directions, as illustrated in Fig. 7.

B. Performance Comparison and Convergence Analysis

Fig. 8 presents the learning curves of SSAC-MGI compared with all aforementioned unconstrained and constrained RL algorithms in a scenario involving three UAVs. Experimental results

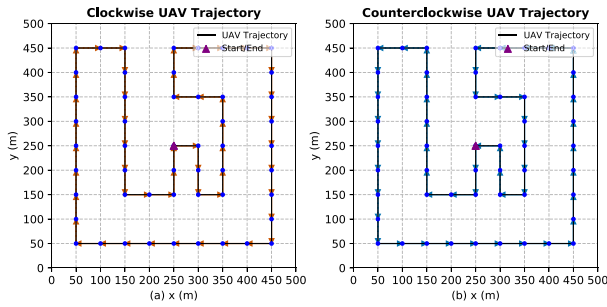


Fig. 7. Two manual UAV trajectories [18].

across two random seeds demonstrate that SSAC-MGI consistently outperforms all baseline algorithms by achieving the highest cumulative rewards and the lowest safety violation costs on both trace datasets. Moreover, it exhibits faster convergence and overall enhanced performance. These improvements are gained owing to the ability of our algorithm to decouple the objectives of reward maximization and safety cost minimization via a two-agent Markov game intervention model. Further analysis of the training reward curves in Fig. 8 reveals that the cumulative reward per episode for V2017 (in subfigure (a)) is higher than that for V2018 (in subfigure (c)). This observation can be explained by two key differences: (i) Jobs in V2017 show a more balanced distribution across resource types, as illustrated in Fig. 5(a); (ii) Compared to V2018, most jobs in V2017 have looser and more uniformly distributed deadlines, as shown in Fig. 6(b) and (c).

We also present the convergence curves of key evaluation metrics observed during the training process with the proposed SSAC-MGI algorithm. The results are compared with those of the MANUAL policy based on the system objectives defined in Equation (23), including the job miss rate, UAV energy cost (i.e., the normalized average energy consumption per UAV movement per time slot), and UE energy cost (i.e., the normalized average energy consumption per job uploaded by UEs per time slot), and the safety metric defined in Equation (26). Fig. 9 shows that all evaluation metrics under the SSAC-MGI algorithm gradually stabilize within a bounded range across both datasets, demonstrating convergence during training. Under the configuration with two manually predefined and synchronized flight trajectories in clockwise and counter-clockwise directions, the system achieves full spatial coverage, the lowest job miss rate, and the lowest average energy consumption on the UE side. However, even with only three UAVs deployed, this manual strategy suffers from elevated safety risks and significantly higher energy consumption on the UAV side. In contrast, the proposed SSAC-MGI algorithm incurs negligible safety cost and demonstrates significantly lower UAV energy consumption, thereby ensuring system safety throughout training and deployment. While the job miss rate achieved by our algorithm is slightly higher than that achieved by the manual policy, it can be effectively mitigated by deploying a larger number of UAVs.

C. Performance Evaluation With Varying Numbers of UAVs

The results shown in Figs. 10 and 11 are averaged over 100 testing episodes for each algorithm under different UAV configurations, with the number of UAVs ranging from 2 to 6.

To evaluate the overall performance of the proposed algorithm with respect to the three considered key metrics, we define a utility function as $f(u) = 1 - (w_1 r_1 + w_2 r_2 + w_3 r_3)$, where r_1 , r_2 , and r_3 represent the job miss rate, UAV energy cost, and UE energy cost, respectively. The weights are set equally, i.e., $w_1 = w_2 = w_3 = \frac{1}{3}$. Based on the defined utility function, lower values of r_1 , r_2 , and r_3 lead to a higher utility score, indicating better overall performance.

In both Figs. 10 and 11, we observe that increasing the number of UAVs generally improves the overall system performance. This aligns with the intuitive understanding that deploying more UAVs enhances network service capacity and reduces the overall job miss rate. However, this improvement comes at the cost of increased safety violation risks. As the number of UAVs increases, all baseline algorithms experience a rise in total safety violation costs, primarily due to the higher likelihood of in-flight collisions with obstacles or other UAVs. In terms of safety performance, SSAC-MGI consistently outperforms all baselines across different configurations, exhibiting significantly lower safety costs. In contrast, MANUAL shows a steep increase in safety violation costs as the number of UAVs increases, while its utility scores remain relatively stagnant. This is mainly because MANUAL focuses solely on achieving full UE coverage. Even though we manually define two opposite-direction trajectories to mitigate conflicts, collisions still occur when UAVs remain stationary to handle multiple incoming job requests, as in the case of two UAVs. Furthermore, SSAC-MGI demonstrates stable and robust performance across both dataset distributions. In the V2018 dataset, where tasks are shorter and deadlines are tighter, UAVs must move more frequently to handle rapid job arrivals. This increased movement leads to relatively higher safety violation costs and slightly reduced utility scores, especially in scenarios with fewer UAVs.

Figs. 12 and 13 present the results of three evaluation metrics under the proposed SSAC-MGI algorithm, with the number of UAVs ranging from 2 to 6 and values averaged over 100 testing episodes on both datasets. The energy consumption metrics denote the average energy usage per UAV and per UE's uploading request during each time slot, respectively. As shown in both figures, increasing the number of UAVs consistently reduces the system's job miss rate, which is the primary factor contributing to the improvement in utility scores. Consistent with previous observations, the job miss rate in dataset V2018 remains higher than that in V2017, as its tasks generally require more concentrated resource types and have shorter execution durations. Another consistent finding from both figures is the trade-off between UAV and UE energy consumption, i.e., relatively higher UAV energy consumption is usually associated with lower UE energy consumption. This occurs because UE transmission energy is passively influenced by channel conditions, which depend on UAV proximity, coverage radius, available resource types, and the UE's transmit power constraint. Since UAV and UE energy consumption are inherently coupled, reducing UAV mobility to save energy may increase UE energy consumption, as users are forced to transmit over longer distances with higher transmit power due to suboptimal UAV positioning. Conversely, allowing UAVs to consume more energy through proactive repositioning can alleviate the energy burden on UEs by improving channel quality and reducing transmission distance.

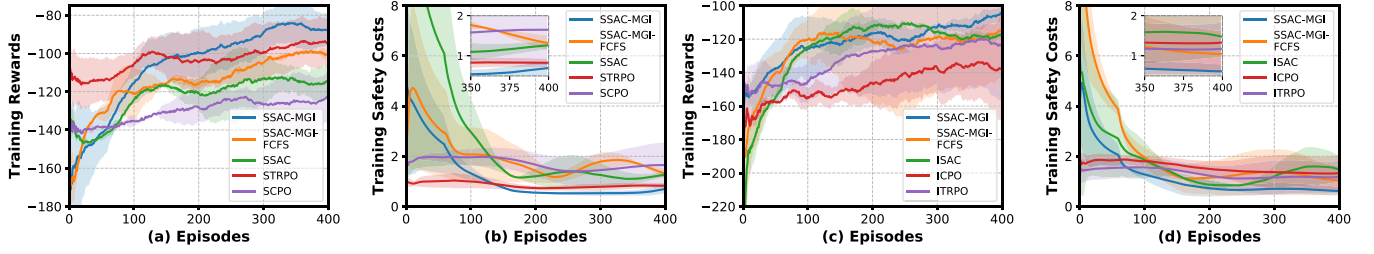


Fig. 8. Training rewards and training safety costs in datasets V2017 (left (a) and (b)) and V2018 (right (c) and (d)).

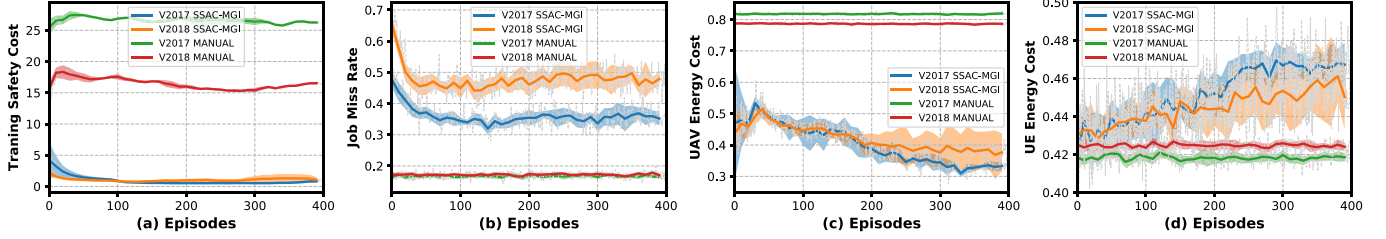


Fig. 9. Performance comparison between SSAC-MGI and the manual policy in terms of safety cost, job miss rate, average UAV energy consumption, and average UE uploading energy consumption on two datasets.

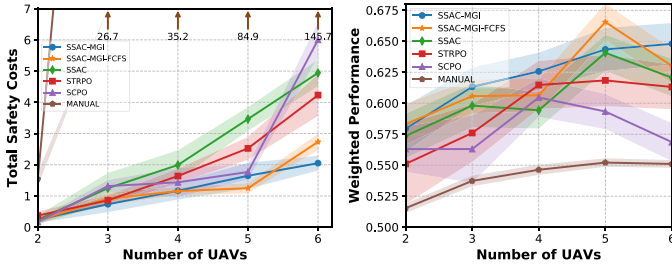


Fig. 10. Performance comparison during testing on V2017 dataset: (a) total safety cost; (b) weighted utility of job miss rate and energy cost.

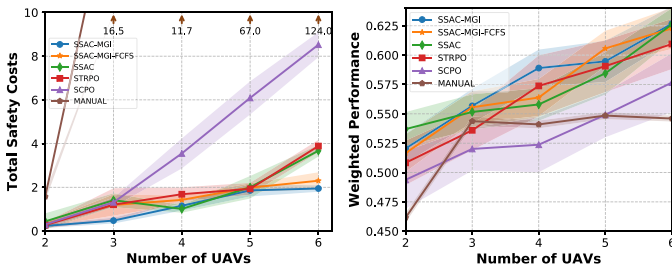


Fig. 11. Performance comparison during testing on V2018 dataset: (a) total safety cost; (b) weighted utility of job miss rate and energy cost.

D. Scalability Evaluation Under Varying Numbers of UAVs and Three Channel Models

We evaluate the scalability of the SSAC-MGI algorithm under varying UAV numbers and fading channel models by expanding the MECN area from $500\text{m} \times 500\text{m}$ with 2–6 UAVs and 20 UEs to $1000\text{m} \times 1000\text{m}$ with 8–12 UAVs and 50 UEs, and by employing a large-scale path-loss model, a practical Rician fading model, and a Rayleigh fading model. The results shown

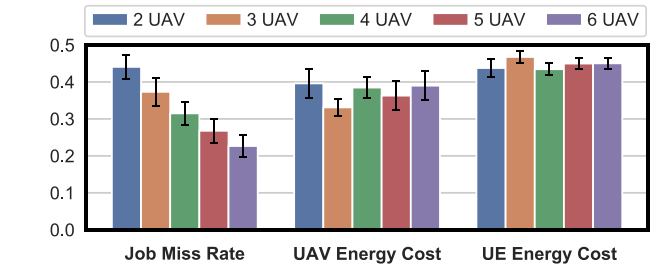


Fig. 12. SSAC-MGI performance on V2017 with different number of UAVs.

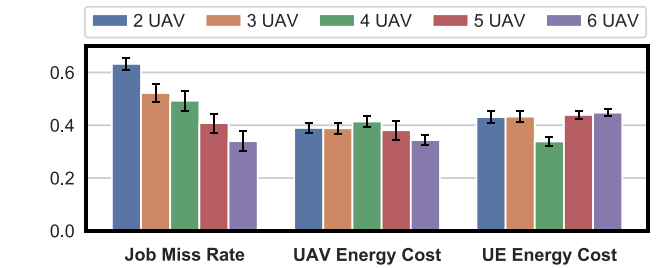


Fig. 13. SSAC-MGI performance on V2018 with different number of UAVs.

in Table IV for each average performance metric are collected from the 500-th to the 600-th training episodes.

For dataset V2017, we set the Rician factor to $\kappa=0$, thereby reducing the channel model to Rayleigh fading. For dataset V2018, following the configuration in [27], we adopt $\kappa=4$. Although these datasets employ different fading models, their performance trends are consistent, allowing us to discuss them jointly. First, across both datasets, increasing the UAV count raises the total safety cost while lowering the job-miss rate, corroborating our findings in Subsection VI-C. Second, expanding

TABLE IV
SCALABILITY OF SSAC-MGI WITH A VARYING NUMBER OF UAVS AND THREE CHANNEL MODELS

Dataset	UAV	Path-loss fading channel model					Rayleigh (V2017) / Rician (V2018) fading channel model				
		SC ($\downarrow$)	Job MR ($\downarrow$)	UAV EC ($\downarrow$)	UE EC ($\downarrow$)	CPU-U ($\uparrow$)	SC ($\downarrow$)	Job MR ($\downarrow$)	UAV EC ($\downarrow$)	UE EC ($\downarrow$)	CPU-U ($\uparrow$)
V2017	8	4.45 $\pm$ 0.05	0.39 $\pm$ 0.03	0.42 $\pm$ 0.01	0.44 $\pm$ 0.01	0.82 $\pm$ 0.02	4.59 $\pm$ 0.04	0.39 $\pm$ 0.03	0.43 $\pm$ 0.01	0.95 $\pm$ 0.05	0.83 $\pm$ 0.01
	9	6.43 $\pm$ 0.02	0.38 $\pm$ 0.03	0.42 $\pm$ 0.01	0.44 $\pm$ 0.01	0.82 $\pm$ 0.02	6.13 $\pm$ 0.06	0.39 $\pm$ 0.03	0.43 $\pm$ 0.01	0.94 $\pm$ 0.05	0.82 $\pm$ 0.01
	10	6.81 $\pm$ 0.13	0.36 $\pm$ 0.02	0.42 $\pm$ 0.01	0.44 $\pm$ 0.01	0.82 $\pm$ 0.01	7.25 $\pm$ 0.06	0.35 $\pm$ 0.03	0.43 $\pm$ 0.01	0.93 $\pm$ 0.04	0.82 $\pm$ 0.02
	11	9.22 $\pm$ 0.14	0.35 $\pm$ 0.03	0.43 $\pm$ 0.01	0.44 $\pm$ 0.01	0.81 $\pm$ 0.02	10.17 $\pm$ 0.14	0.33 $\pm$ 0.03	0.44 $\pm$ 0.01	0.93 $\pm$ 0.05	0.81 $\pm$ 0.01
	12	13.07 $\pm$ 0.32	0.34 $\pm$ 0.03	0.43 $\pm$ 0.01	0.44 $\pm$ 0.01	0.81 $\pm$ 0.01	9.97 $\pm$ 0.08	0.31 $\pm$ 0.03	0.43 $\pm$ 0.01	0.93 $\pm$ 0.04	0.82 $\pm$ 0.02
V2018	8	4.85 $\pm$ 0.12	0.57 $\pm$ 0.03	0.41 $\pm$ 0.01	0.44 $\pm$ 0.01	0.76 $\pm$ 0.02	4.77 $\pm$ 0.13	0.56 $\pm$ 0.03	0.42 $\pm$ 0.01	0.66 $\pm$ 0.03	0.77 $\pm$ 0.02
	9	6.66 $\pm$ 0.23	0.53 $\pm$ 0.03	0.42 $\pm$ 0.01	0.44 $\pm$ 0.02	0.76 $\pm$ 0.02	5.95 $\pm$ 0.08	0.52 $\pm$ 0.03	0.42 $\pm$ 0.01	0.66 $\pm$ 0.03	0.77 $\pm$ 0.02
	10	7.09 $\pm$ 0.12	0.52 $\pm$ 0.03	0.42 $\pm$ 0.01	0.44 $\pm$ 0.01	0.75 $\pm$ 0.02	6.79 $\pm$ 0.12	0.52 $\pm$ 0.03	0.42 $\pm$ 0.01	0.66 $\pm$ 0.03	0.76 $\pm$ 0.02
	11	8.81 $\pm$ 0.08	0.47 $\pm$ 0.04	0.42 $\pm$ 0.01	0.44 $\pm$ 0.01	0.74 $\pm$ 0.02	10.56 $\pm$ 0.18	0.50 $\pm$ 0.03	0.42 $\pm$ 0.01	0.65 $\pm$ 0.03	0.75 $\pm$ 0.02
	12	9.49 $\pm$ 0.11	0.45 $\pm$ 0.04	0.42 $\pm$ 0.01	0.44 $\pm$ 0.01	0.74 $\pm$ 0.01	11.79 $\pm$ 0.29	0.47 $\pm$ 0.04	0.42 $\pm$ 0.01	0.65 $\pm$ 0.02	0.75 $\pm$ 0.01

Annotation: SC: safety cost; Job MR: job-miss rate; UAV EC: UAV energy cost; UE EC: UE energy cost; CPU-U: CPU utilization.

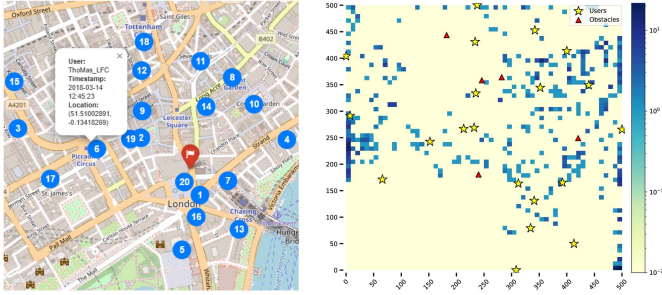


Fig. 14. UAV-learned trajectory distribution with static UEs.

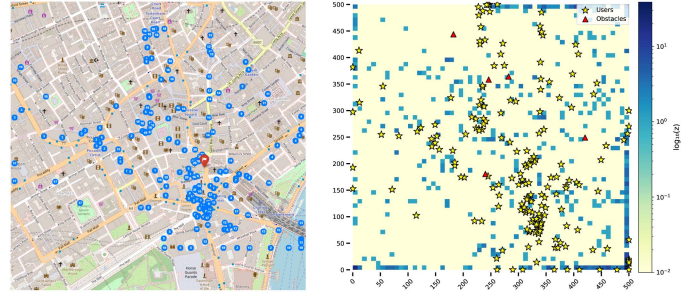


Fig. 15. UAV-learned trajectory distribution with mobile UEs.

the MECN area increases the average UAV–UE travel distance while also resulting in a sparser UE distribution, causing a slight rise in the average energy cost of UAVs and UEs. Furthermore, the additional UAVs deployed in larger scales provide sufficient computational capacity, so the average energy cost between UAVs and UEs remains nearly unchanged, making the energy trade-off negligible. Third, despite the variability introduced by Rician and Rayleigh fading, SSAC-MGI adapts to these stochastic channels and sustains safety costs and job-miss rates comparable to those under the deterministic path-loss model. With this set of experimental parameters, for all three fading models, the measured average UE energy cost follows the theoretical ordering $\bar{p}_{PL} < \bar{p}_{Ric} < \bar{p}_{Ray}$ established in *Lemma 2*, indicating that the UE requires the larger transmit power to guarantee the successful request uploading under fading channels, compared with that under the path loss channel. In summary, these consistent trends across varying numbers of UAVs and three fading channel models in a larger-scale MECN validate the scalability and robustness of SSAC-MGI.

E. Analysis of UAV-Learned Trajectory Distributions With Mobile/Static UEs and Static Obstacles

We conduct a detailed analysis of the trajectory distributions learned by SSAC-MGI in a representative scenario comprising 3 UAVs and 20 UEs, incorporating both static and mobile UEs. To simulate mobility diversity and positioning errors, small Gaussian noise is added to introduce controlled randomness. This analysis provides a visualization of UAVs' safety-aware trajectories.

Figs. 14 and 15 present the resulting UAV trajectory distributions under a 3-UAV setting on the V2017 dataset. In each figure,

the left panel shows UE locations overlaid on a Google Map, while the right panel projects these positions onto a 500×500 2D coordinate plane. UAV trajectories over 300 time slots are visualized using heatmaps, with color intensity representing the frequency of UAV presence. To better highlight the distributional differences in UAV locations, we apply a logarithmic transformation ($\log_{10} z$) to the visit frequency, where z denotes the cumulative number of UAV visits to each location duration the mission. Yellow pentagrams indicate UEs, while red triangles represent static obstacles. The results in Fig. 14 illustrate that, under static UEs, the UAVs tend to align their trajectories closely with the UE locations, despite not having prior knowledge of their positions. Moreover, the UAV trajectories consistently avoid the red static obstacles, demonstrating effective obstacle-aware planning. Furthermore, Fig. 15 illustrates the scenario in which each UE moves randomly within the network. The results demonstrate that the learned UAV trajectories effectively adapt to the spatial dynamics of mobile UEs, achieving both adaptive coverage and reliable obstacle avoidance. In addition, the accumulated safety violation costs in Fig. 10 are below 1 when the number of UAVs is 3, which indicates effective collision avoidance among UAVs. This indicates strong adaptability of the proposed method to varying UE mobility patterns, while ensuring safe, goal-directed trajectory planning.

VII. CONCLUSION AND FUTURE WORK

In this work, we investigated the cooperative trajectory optimization of heterogeneous UAVs in realistic MEC environments characterized by uncertain UE locations and dynamic computation request arrivals. We propose SSAC-MGI, a multi-agent safe reinforcement learning framework designed to generate

environment-adaptive trajectory planning policies. It incorporates a fine-grained onboard resource allocation mechanism to ensure both trajectory safety and efficient resource utilization in heterogeneous UAV systems operating under stochastic conditions. To evaluate our approach, we construct a realistic UAV-enabled MEC environment using actual user distributions and real-world Alibaba job traces. Extensive simulations demonstrate that our method significantly outperforms existing baselines in terms of job miss rate and average energy consumption of both UAVs and UEs, while maintaining safety under unseen scenarios.

For real-world deployment of such MECN, achieving high-fidelity environmental sensing, including precise UE localization and accurate channel state estimation, is critical, as delays or errors can degrade trajectory planning performance. Compared with large-scale static UAV deployments, which necessitate numerous drones stationed at fixed locations, our approach enables dynamic coverage using substantially fewer units. Nonetheless, ensuring absolute safety requires not only guidance from our scheduling policy but also real-time perception and online fine-tuning. To address these requirements, future work will explore multi-modal learning techniques for real-time perception, integrating image-based UE localization, obstacles detection, and analysis of historical trajectory information flows to provide richer observations and more informed trajectory guidance. Near-range UAV identification will further enable finer-grained trajectory adjustments. The current work serves as a preliminary exploration in this direction.

ILLUSTRATION OF MAIN TECHNICAL TERMS AND ACRONYMS

- *Mobile edge computing networking (MECN)*: A network consisting of user equipment (mobile/static) and edge servers that relocates cloud-level computing, storage, and networking capabilities from the core network to the radio-access edge, thereby reducing end-to-end latency, alleviating back-haul congestion, and enabling context-aware, real-time services for 5/6G applications.
- *Unmanned aerial vehicle (UAV)*: In this work, the UAV is regarded as the mobile edge server equipped with dedicated computational resources.
- *User equipment (UE)*: UE refers to a service objective by UAV, which randomly generates job requests with diverse resource demands during the network operation.
- *Decentralized partially observable Markov decision process (Dec-POMDP)*: The Dec-POMDP model extends single-agent POMDP models by considering joint actions and observations. POMDP extends the MDP model by incorporating observations and their probability of occurrence conditional on the state of the environment.
- *Decentralized training and decentralized execution (DTDE)*, *Centralized training and decentralized execution (CTDE)*: MARL is commonly categorized as either centralized or decentralized. Centralized approaches learn one policy to emit joint actions for all agents, whereas DTDE let each agent optimize its own reward. CTDE assumes a training stage during which the learning algorithm can

access data from all agents to learn decentralized (locally-executable) agent policies.

- *Soft actor-critic (SAC)*, *Trust Region Policy Optimization (TRPO)*, *constrained policy optimization (CPO)*: All the three are policy-gradient methods aiming at stable return gains: SAC boosts sample efficiency via maximum-entropy reward shaping; TRPO enforces monotonic improvement with a KL trust-region constraint; CPO layers cost bounds on TRPO to guarantee safety.

REFERENCES

- [1] S. A. Noghabi, L. Cox, S. Agarwal, and G. Ananthanarayanan, "The emerging landscape of edge computing," *GetMobile, Mobile Comput. Commun.*, vol. 23, no. 4, pp. 11–20, 2020.
- [2] L. Lin, L. Pan, and S. Liu, "SpotDAG: An RL-based algorithm for DAG workflow scheduling in heterogeneous cloud environments," *IEEE Trans. Serv. Comput.*, vol. 17, no. 5, pp. 2904–2917, Sep./Oct. 2024.
- [3] S. Mittal, "A survey on optimized implementation of deep learning models on the NVIDIA Jetson platform," *J. Syst. Archit.*, vol. 97, pp. 428–442, 2019.
- [4] F. Zhou, Y. Wu, R. Q. Hu, and Y. Qian, "Computation rate maximization in UAV-enabled wireless-powered mobile-edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 9, pp. 1927–1941, Sep. 2018.
- [5] X. Hu, K.-K. Wong, K. Yang, and Z. Zheng, "UAV-assisted relaying and edge computing: Scheduling and trajectory optimization," *IEEE Trans. Wireless Commun.*, vol. 18, no. 10, pp. 4738–4752, Oct. 2019.
- [6] M. Li, N. Cheng, J. Gao, Y. Wang, L. Zhao, and X. Shen, "Energy-efficient UAV-assisted mobile edge computing: Resource allocation and trajectory optimization," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3424–3438, Mar. 2020.
- [7] X. Gao and L. Zhai, "Service experience oriented cooperative computing in cache-enabled UAVs assisted MEC networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9721–9736, Oct. 2024.
- [8] T. Zhang, Y. Xu, J. Loo, D. Yang, and L. Xiao, "Joint computation and communication design for UAV-assisted mobile edge computing in IoT," *IEEE Trans. Ind. Informat.*, vol. 16, no. 8, pp. 5505–5516, Aug. 2020.
- [9] J. Du, T. Lin, C. Jiang, Q. Yang, C. F. Bader, and Z. Han, "Distributed foundation models for multi-modal learning in 6G wireless networks," *IEEE Wireless Commun.*, vol. 31, no. 3, pp. 20–30, Jun. 2024.
- [10] X. Liu, Y. Liu, Y. Chen, and L. Hanzo, "Trajectory design and power control for multi-UAV assisted wireless networks: A machine learning approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 8, pp. 7957–7969, Aug. 2019.
- [11] Y. Miao, K. Hwang, D. Wu, Y. Hao, and M. Chen, "Drone swarm path planning for mobile edge computing in industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 5, pp. 6836–6848, May 2023.
- [12] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Sep. 2022.
- [13] C. H. Liu, X. Ma, X. Gao, and J. Tang, "Distributed energy-efficient multi-UAV navigation for long-term communication coverage by deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 19, no. 6, pp. 1274–1285, Jun. 2020.
- [14] Y. Zhu, M. Chen, S. Wang, Y. Hu, Y. Liu, and C. Yin, "Collaborative reinforcement learning based unmanned aerial vehicle (UAV) trajectory design for 3D UAV tracking," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 10787–10802, Dec. 2024.
- [15] Z. Hu, F. Zeng, Z. Xiao, B. Fu, H. Jiang, and H. Chen, "Computation efficiency maximization and QOE-provisioning in UAV-enabled MEC communication systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1630–1645, Apr.–Jun. 2021.
- [16] L. Spampinato, D. Ferretti, C. Buratti, and R. Marini, "Joint trajectory design and radio resource management for UAV-aided vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 74, no. 1, pp. 847–860, Jan. 2025.
- [17] R. Zhou, X. Wu, H. Tan, and R. Zhang, "Two time-scale joint service caching and task offloading for UAV-assisted mobile edge computing," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2022, pp. 1189–1198.
- [18] P. Wang et al., "Decentralized navigation with heterogeneous federated reinforcement learning for UAV-enabled mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13621–13638, Dec. 2024.

- [19] Z. Ning, Y. Yang, X. Wang, Q. Song, L. Guo, and A. Jamalipour, "Multi-agent deep reinforcement learning based UAV trajectory optimization for differentiated services," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5818–5834, May 2024.
- [20] S. Yin and F. R. Yu, "Resource allocation and trajectory design in UAV-aided cellular networks based on multiagent reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2933–2943, Feb. 2022.
- [21] F. Song et al., "Evolutionary multi-objective reinforcement learning based trajectory control and task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7387–7405, Dec. 2022.
- [22] H. Hao, C. Xu, W. Zhang, S. Yang, and G.-M. Muntean, "Joint task offloading, resource allocation, and trajectory design for multi-UAV cooperative edge computing with task priority," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8649–8663, Sep. 2024.
- [23] Y. Sixing and F. Yu, "Resource allocation and trajectory design in UAV-aided cellular networks based on multiagent reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2933–2943, Feb. 2022.
- [24] Alibaba, "Alibaba cluster trace program," 2022, Accessed: Nov. 31, 2022. [Online]. Available: <https://github.com/alibaba/clusterdata/tree/v2018>.
- [25] Y. Wang, Z.-Y. Ru, K. Wang, and P.-Q. Huang, "Joint deployment and task scheduling optimization for large-scale mobile users in multi-UAV-enabled mobile edge computing," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3984–3997, Sep. 2020.
- [26] H. Hydder, D. N. K. Jayakody, K. T. Hemachandra, and T. Samarasinghe, "UAV deployment for data collection in energy constrained WSN system," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2022, pp. 1–6.
- [27] Y. Han, L. Liu, L. Duan, and R. Zhang, "Towards reliable UAV swarm communication in D2D-enhanced cellular networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 3, pp. 1567–1581, Mar. 2021.
- [28] N. Lin, Y. Fan, L. Zhao, X. Li, and M. Guizani, "Green: A global energy efficiency maximization strategy for multi-UAV enabled communication systems," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7104–7120, Dec. 2023.
- [29] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.
- [30] X. Zhang, R. Jia, Q. Yin, Z. Zheng, and M. Li, "Intelligent trajectory design and charging scheduling in wireless rechargeable sensor networks with obstacles," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8664–8679, Sep. 2024.
- [31] F. A. Oliehoek et al., *A Concise Introduction to Decentralized POMDPs*, vol. 1. Berlin, Germany: Springer, 2016.
- [32] E. Altman, "Constrained Markov decision processes with total cost criteria: Lagrangian approach and dual linear program," *Math. Methods Operations Res.*, vol. 48, pp. 387–417, 1998.
- [33] A. Hans, D. Schneegaß, A. M. Schäfer, and S. Udluft, "Safe exploration for reinforcement learning," in *Proc. Euro. Symp. Artif. Neural Netw.*, 2008, pp. 143–148.
- [34] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [35] T. Haarnoja et al., "Soft actor-critic algorithms and applications," 2018, *arXiv:1812.05905*.
- [36] J. Duan, Y. Guan, S. E. Li, Y. Ren, Q. Sun, and B. Cheng, "Distributional soft actor-critic: Off-policy reinforcement learning for addressing value estimation errors," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 11, pp. 6584–6589, Nov. 2022.
- [37] D. Mguni et al., "DESTa: A framework for safe reinforcement learning with Markov games of intervention," 2023, *arXiv:2110.14468*.
- [38] Twitter, "Twitter-dataset," 2018. [Online]. Available: <https://github.com/pswf/Twitter-Dataset/blob/master/Dataset>
- [39] Q. Liu and Z. Yu, "The elasticity and plasticity in semi-containerized co-locating cloud workload: A view from Alibaba trace," in *Proc. ACM Symp. Cloud Comput.*, 2018, pp. 347–360.
- [40] R. Jia, Q. Fu, Z. Zheng, G. Zhang, and M. Li, "Energy and time trade-off optimization for multi-UAV enabled data collection of IoT devices," *IEEE/ACM Trans. Netw.*, vol. 32, no. 6, pp. 5172–5187, Dec. 2024.
- [41] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1889–1897.
- [42] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 22–31.



Xiuling Zhang received the MS degree in software engineering from Zhejiang Normal University, China, in 2022. She is currently working toward the PhD degree with the College of Systems Engineering, National University of Defense Technology, Changsha, China. From 2023 to 2025, she was a visiting student with the Engineering Systems and Design Pillar, Singapore University of Technology and Design, Singapore. Her current research interests include mobile edge computing, smart IoT, and reinforcement learning.



Riheng Jia (Member, IEEE) received the BE degree in electronics and information engineering from the Huazhong University of Science and Technology, China, in 2012, and the PhD degree in computer science and technology from Shanghai Jiao Tong University, Shanghai, China, in 2018. He is currently an associate professor with the School of Computer Science and Technology, Zhejiang Normal University, China. His current research interests include wireless networks, energy harvesting networks, and smart IoT.



Quanjun Yin was born in Hunan, China, in 1978. He received the BS, MS, and PhD degrees in simulation engineering from the College of Systems Engineering, National University of Defense Technology, Changsha, China, in 2008. His research interests include cognitive process modeling, qualitative spatial reasoning and planning, cooperation and negotiation, cloud-based simulation, and edge computing.



Zhonglong Zheng (Member, IEEE) received the BE degree from the China University of Petroleum, China, in 1999, and the PhD degree from Shanghai Jiao Tong University, China, in 2005. He is currently a full professor with the School of Computer Science and Technology, Zhejiang Normal University, China. His research interests include machine learning, computer vision, and blockchain.



Minglu Li (Fellow, IEEE) received the PhD degree in computer software from Shanghai Jiao Tong University in 1996. He is currently a full professor and the director of Artificial Intelligence Internet of Things Center, Zhejiang Normal University. He is also the director of Network Computing Center, Shanghai Jiao Tong University. He has authored or coauthored more than 400 papers in academic journals and international conferences. His research interests include vehicular networks, Big Data, cloud computing, and wireless sensor networks. He was the chairman of

Technical Committee on Services Computing from 2004 to 2016 and Technical Committee on Distributed Processing from 2005 to 2017, of IEEE Computer Society in Great China region. He was the general co-chair of IEEE SCC, IEEE CCGrid, IEEE ICPADS, and IEEE IPDPS, and the vice chair of IEEE INFOCOM. He was also a PC member of more than 50 international conferences including IEEE INFOCOM 2009–2016 and IEEE CCGrid 2008.